Counterexamples in Type Systems

collated by Stephen Dolan, with thanks to Andrej Bauer, Leo White and Jeremy Yallop

A compendium of horrible programs that crash, segfault or otherwise explode.

counterexamples.org

Contents

Introduction

Welcome to *Counterexamples in Type Systems*, a compendium of horrible programs that crash, segfault or otherwise explode.

The "counterexamples" here are programs that go wrong in ways that should be impossible: corrupt memory in Rust, produce a `ClassCastException` in cast-free Java, segfault in Haskell, and so on. This book is a collection of such counterexamples, each with some explanation of what went wrong and references to the languages or systems in which the problem occurred.

It's intended as a resource for researchers, designers and implementors of static type systems, as well as programmers interested in how type systems fit together (or don't).

Type systems

For the purposes of this book, a "type system" is a system that does local checks of a program's source code, in order to establish some global property about the program's behaviour. The local checks usually assign a type to each variable and verify that they are used only in contexts appropriate to their type, while the global property varies but is generally some sort of safety property: the absence of memory corruption, use-after-free errors, and the like.

This is intentionally a fairly narrow definition. Some examples of things that are not "type systems":

- **Linters**: Linters do local checks of a program's source code, but don't try to establish any global property. Just because a program passes a linter's checks does not imply anything definite about its behaviour.
- **Python's type system**: Dynamic languages like Python do local type checks to establish global safety properties. However, the type checks are not done purely on the program's source code before running it, but examine the actual runtime inputs as well.
- **C++'s type system**: C++ (among other unsafe languages) has a type system that does many local checks of a program's source code. However, passing these checks does not establish anything definite about a program's behaviour: many C++ programs pass all of the compiler's type checking, and yet exhibit use-after-free errors, memory corruption, and arbitrary behaviour when run.
- **Some program analysis tools**: Some static program analysis tools do only global checks (requiring the whole program's source code to be available at once), while others detect certain issues but do not attempt to establish any global property when no issues are found. Still others do fit the definition of "type system", though: the line here is blurry.

This is not to say that there's something wrong with the above tools, but they're not what this book is about. Some examples of things that do fit the definition are the type systems of languages like Java, Haskell and Rust.

Soundness and counterexamples

Type systems make the claim that any program that passes the local type checks will definitely have the global safety properties. A type system is *sound* if this claim is true, and to say that a system is *unsound* is to say that there exists some program which passes all type checks, and yet whose behaviour violates the global safety properties (corrupts memory, crashes unexpectedly, etc.). A *counterexample* is such a program, exhibiting a *soundness bug*.

Different languages enforce different properties, so exactly what "soundness" means varies somewhat. For instance, a global property established by the Coq type system is that all programs halt. So, a program that looped infinitely would be a counterexample in Coq, but not in Java, which makes no such claim.

The goal of this book is to collect such counterexamples, especially those that crop up in similar forms across multiple languages. These fall into a couple of broad categories:

- **Missing checks**: The simplest sort of soundness bug is when some important type check is missing from the type checker. Usually, these are straightforward bugs, easily fixed once discovered, highly specific to a particular compiler, and not very interesting. However, certain missing checks are common mistakes that keep coming up across many different type checkers and languages, and a few of these are included here: for instance, missing variance checks (§3) or scope checks (§24).

- **Feature interactions**: Often, two type system features which are perfectly sound when used in isolation have some tricky interaction when used together. Examples of this include mixtures of polymorphism and mutability (§1) or polymorphism and overloading (§12) or intersections and mutability (§28).

- **Subtle differences**: If two parts of a type system use different but similar versions of the same concept, the gap between them can often cause soundness bugs. For instance, does "private" mean private to an object or private to a class (§18)? Does "unequal" mean "not provably equal" or "provably distinct" (§13)?

Organisation (or lack thereof)

Since so many of the counterexamples here depend on interactions between disparate parts of a complex type system, it would be tricky to separate them all into distinct coherent topics, and no attempt to do so has been made.

Instead, each entry is tagged with the type system features involved, and the index and glossary at the end of the book lists the features and the counterexamples in which they appear.

The entries themselves are mostly independent, and it should be possible to read them in any order. However, a few of them do refer to previous entries, and some attempt was made to put the simpler entries earlier, so if you do intend to read the whole thing then going start to finish is best.

A note on sources

While much type system research appears in formal academic publications, it is relatively rare (but not unheard of!) to publish a paper about an unsound system.

So, as well as academic papers, many of the counterexamples here are drawn from unpublished notes, the archives of the TYPES forum, blogs by programming language researchers, designers and implementors, and language-specific bugtrackers, forums and mailing lists. A major reason for the existence of this book is to collect these various sources together.

In particular, compiler bugtrackers are a major source of the counterexamples reproduced here. Try not to read too much into how often different bugtrackers show up: while it is tempting to assume that the more often a compiler appears, the buggier it is, this would be wrong for several reasons:

- the author works professionally on OCaml, so it is somewhat over-represented due to familiarity,
- the same applies to other languages that the author likes or at least vaguely understands, and
- languages whose community does a good job of publicly tracking, explaining, and writing up issues that arise are also over-represented.

1 Polymorphic references

Polymorphism • Mutation

Polymorphism, as found in ML and Haskell (and in C# and Java where it's called "generics") lets the same definition be reused with multiple different unrelated types. For instance, here's a polymorphic function that creates a single-element list:

OCaml

```
let singleton x = [x]
```

Java

```
static <A> List<A> singleton(A x) {
    return Arrays.asList(x);
}
```

The same `singleton` function can be used to turn an integer into a list of integers, and also to turn a string into a list of strings.

The standard typing rule for polymorphism is:

$$\frac{\Gamma \vdash e : A}{\Gamma \vdash e : \forall \alpha . A} \text{ if } \alpha \text{ not free in } \Gamma$$

That is, if an expression e has a type A which mentions a type variable α, and the type variable α is used only in the type A, then the value of e can be reused with different types substituted for α. The justification is that different uses of e can use different types for α, since nothing outside the type of e depends on which type is chosen. This process is called *generalising* the type variable α.

But if e can allocate mutable state (say, a mutable reference cell, or a mutable array), then different uses of e can share state:

OCaml

```
let r = ref [] in           (* r : ∀ α . α list ref *)
r := [0];                   (* using r as an int list ref *)
print_string (List.hd !r)   (* using r as a string list ref *)
(* Crash! (Well, it would, if the OCaml compiler allowed this) *)
```

One obvious-but-broken solution is to disallow polymorphism when the type of e mentions types with mutable state. This doesn't work: because functions can hide types in their closures, the problem occurs even when the type of e doesn't mention mutable state:

OCaml

```
let f =
  let r = ref None in
  fun x ->
    match !r with
```

```
    | None -> r := Some x; x
    | Some x -> x
(* The value restriction means f is not polymorphic,
   but if it were then this would crash: *)
let _ =
  let _ = f 42 in
  print_string (f "hello")
```

This problem originated in the ML family of languages, as these were the first to combine mutable references and polymorphism. A related problem appeared in Standard ML of New Jersey's implementation of `call/cc`[1], which like mutable references allows multiple uses of an expression to share state (by sharing the continuation). Recently, an instance of this problem arose in OCaml, due to an incorrect typechecker refactoring[2]. The problem has also appeared in Elm[3], using channels to provide the shared mutable state.

Standard ML

```
(* Counterexample by Bob Harper and Mark Lillibridge *)
fun left (x,y) = x;
fun right (x,y) = y;

let val later = (callcc (fn k =>
    (fn x => x,
         fn f => throw k (f, fn f => ()))))
in
    print (left(later)"hello world!\n");
    right(later)(fn x => x+2)
end
```

OCaml

```
(* Counterexample by Thierry Martinez *)
let f x =
  let ref : type a . a option ref = ref None in
  ref := Some x;
  Option.get !ref

let () = print_string (f 0)
```

Elm

```
-- Counterexample by Izaak Meckler
import Signal
import Html
import Html.Attributes(style)
import Html.Events(..)
import Maybe
```

[1] ML with callcc is unsound (TYPES mailing list) Bob Harper and Mark Lillibridge (1991)
[2] github.com/ocaml/ocaml/issues/9856 (2020)
[3] github.com/elm/compiler/issues/889 (2015)

```
c = Signal.channel Nothing

s : Signal Int
s = Signal.map (Maybe.withDefault 0) (Signal.subscribe c)

draw x =
  Html.div
  [ onClick (Signal.send c (Just "I am not an Int"))
  , style [("width", "100px"), ("height", "100px")]
  ]
  [ Html.text (toString (x + 1)) ]
  |> Html.toElement 100 100

main = Signal.map draw s
```

There are several solutions:

- **Separate pure and effectful types**

  The original solution chosen in Standard ML distinguishes *applicative* type variables from *imperative* type variables. Only applicative variables can be generalised, and only imperative variables can be used in the type of mutable references. There are several variants of this system, from Tofte's original one to extensions used in SML/NJ by MacQueen and others. Greiner[4] has a survey of the variants.

  This approach has the disadvantage that internal use of mutation cannot be hidden: a polymorphic sorting function that internally uses temporary mutable storage cannot be given the same type as one that does not.

  Leroy and Weis[5] propose a related approach that avoids generalising type variables that may be used in mutable references, without needing to distinguish two classes of type variables. However, to avoid the issue above of functions hiding types in their closure, their solution must distinguish two different sorts of function type according to whether their closure may contain mutable references.

- **The value restriction**

  A simpler solution was proposed by Wright[6] and is now used in most implementations of ML: generalisation of an expression's type only happens if the expression is a *syntactic value*, a class that contains e.g. function definitions and tuple constructions, but not function calls or anything that could possibly allocate a mutable reference.

  This solution is much simpler than the type-based approaches, but in some cases less powerful: notably, a partial application of a curried function cannot be generalised without manually eta-expanding.

[4] Weak Polymorphism Can Be Sound, John Greiner (1996)
[5] Polymorphic type inference and assignment, Xavier Leroy and Pierre Weis (1991)
[6] Simple Imperative Polymorphism, Andrew Wright (1995)

In effect, this is also the solution used by e.g. Java and C#: in these languages, only methods may be given generic types, not fields. This ensures that generalisation only occurs on function definitions.

- **Effect typing**

  Type-and-effect systems have a typing judgement $\Gamma \vdash e : A \,!\, \Delta$, where Δ is the *effect*, representing effects that occur as part of evaluating e. The typing rule for polymorphism in such systems can generalise a type variable α only if it does not appear in Γ *or* Δ. If allocating a new mutable reference of type T results in an effect mentioning T appearing in Δ, then polymorphic mutable references cannot be created.

- **Effect marking**

  Instead of a full-blown type-and-effect system, polymorphic mutable references can be avoided using a type-level marker on computations that may perform any effects at all, and then disabling polymorphism on effectful computations.

  This is the approach taken by Haskell with its monadic encoding of effectful computations in the type `IO a`. Values of this type represent "IO actions" — computations that, when performed, yield results of type `a`, possibly causing some side-effects as well. Polymorphism is available as usual when constructing IO actions, but the type of an IO action's result (as used in `do` notation or via monadic combinators) cannot be generalised. Haskell's `do` notation for monadic computation does support a `let` syntax, but its typing is quite different from ordinary `let`, and it does not allow introducing polymorphism even with an explicit annotation.

  This mechanism allows Haskell to distinguish the following two types:

  Haskell

  ```
  good :: forall a . IO (IORef (Maybe a))
  bad  :: IO (forall a . IORef (Maybe a))
  ```

  Here, `good` is an IO action which allocates a mutable reference of an arbitrary type (fine, implementable as `newIORef Nothing`), while `bad` is an IO action which allocates a polymorphic mutable reference (unsound). Note that the only distinction between these two is the placement of `IO`.

  This reliance on marking `IO` is why Haskell's `unsafePerformIO` is actually unsafe, as it can be used to strip the `IO` marker and create polymorphic mutable references:

  Haskell

  ```
  import Data.IORef
  import System.IO.Unsafe
  badref :: IORef (Maybe a)
  badref = unsafePerformIO (newIORef Nothing)
  main = do
    writeIORef badref (Just 42)
    Just x <- readIORef badref
  ```

```
  putStrLn (x 1)
```

2 Covariant containers

Subtyping • Variance

If there is a subtyping between two types, say that every Car is a Vehicle, then it is natural to extend this subtyping to container types, to say that a List[Car] is also a List[Vehicle].

However, this is only sound for *immutable* List types. If List is mutable, then a List[Vehicle] is something into which I can insert a Bus. If every List[Car] is automatically a List[Vehicle], then you can end up with buses in your list of cars.

This problem occurs with arrays in Java:

Java

```
class Vehicle {}
class Car extends Vehicle {}
class Bus extends Vehicle {}
public class App {
  public static void main(String[] args) {
    Car[] c = { new Car() };
    Vehicle[] v = c;
    v[0] = new Bus(); // crashes with ArrayStoreException
  }
}
```

The solution is to keep track of *variance* (how subtyping of type parameters affects subtyping of the whole type). There are two approaches:

- **Use-site variance** is used in Java (for types other than arrays): a `List<Car>` can never be converted to a `List<Vehicle>`, but can be converted to a `List<? extends Vehicle>`. The elements of a `List<? extends Vehicle>` are known to be `Vehicle`s, but an arbitrary `Vehicle` cannot be inserted into such a list. Each use of `List` specifies how the parameter is allowed to vary.

- **Declaration-site variance** is used in Scala (although use-site variance is also available). This means that the `List` type can be defined as `List[+T]` if immutable, making every `List[Car]` automatically a `List[Vehicle]`. However, if `List` is mutable, it must be defined as `List[T]`, which disables these conversions.

3 Incomplete variance checking

Subtyping • Variance • Typecase • Recursive types

With declaration-site variance (see Covariant containers (§2)), it is possible to define a class `C` with a covariant parameter `A`, so that `C[X]` is a subtype of `C[Y]` whenever `X` is a subtype of `Y`. However, when typechecking `C` it is important to verify that all uses of the parameter `A` are actually compatible with covariance. Many soundness issues have arisen from skipping such checks:

- **Private members** may not need checking for variance, but this is subtle. See Privacy violation (§18).
- **Constructor parameters** do not normally need to be checked for covariance, as they do not form part of an object's interface. However, some language features can expose constructors in the interface, in which case variance checking must be done. In Scala[1], constructors of inner classes are exposed by an object and can refer to the objects' fields, while in Hack[2] constructors are accessible through the (assumed covariant) `classname` type.

Scala

```
// Counterexample by Martin Odersky
class C[+A] {
  private[this] var y: A = _
  def getY: A = y
  class Inner(x: A) {
    y = x
  }
}

object Test {
  def main(args: Array[String]) = {
    val x = new C[String]
    val y: C[Any] = x
    val i = new y.Inner(1)
    val s: String = x.getY
    println(s)
    // Exception in thread "main" java.lang.ClassCastException:
    // java.lang.Integer cannot be cast to java.lang.String
  }
}
```

Hack

```
// Counterexample by Derek Lam (edited)
class Base {}
class Derived extends Base {
```

[1] github.com/scala/bug/issues/9549 (2015)
[2] github.com/facebook/hhvm/issues/7216 (2016)

```
    public function __construct(public int $foo) {}
}
<<__ConsistentConstruct>>
abstract class A<+T> {
    abstract public function __construct(T $v);
}
class B extends A<Derived> {
    public function __construct(Derived $v){echo $v->foo;}
}
class C {
    public static function foo(): void {
        self::bar(B::class);
    }
    public static function bar(classname<A<Base>> $v): void {
        new $v(new Base()); // crashes
    }
}
```

- **GADT parameters** introduce equations between types when matched on, making it unsound to mark them as either covariant or contravariant. This issue showed up in early versions of OCaml and in Scala[3]:

OCaml

```
(* Counterexample by Jeremy Yallop *)
let magic : 'a 'b. 'a -> 'b =
  fun (type a) (type b) (x : a) ->
    let bad_proof (type a) =
      (Refl : (< m : a>, <m : a>) eq :> (<m : a>, < >) eq) in
    let downcast : type a. (a, < >) eq -> < > -> a =
      fun (type a) (Refl : (a, < >) eq) (s : <>) -> (s :> a) in
    (downcast bad_proof ((object method m = x end) :> < >)) # m
```

Scala

```
// Counterexample by Owen Healy
object Test extends App {
  sealed trait Node[+A]
  case class L[C,D](f: C => D) extends Node[C => D]

  def test[A,B](n: Node[A => B]): A => B = n match {
    case l: L[c,d] => l.f
  }

  println {
    test(new L[Int,Int](identity) with
      Node[Nothing]: Node[Int => String])(3): String
  }
}
```

[3] github.com/scala/bug/issues/8563 (2014)

The details of combining GADTs and declaration-site variance are tricky. Giarusso[4] describes the problem in detail, and Scherer and Rémy[5] identify several cases where they can be soundly combined.

- **Self types** allow a class to refer recursively to the type of `this`, which may be a subtype of the class being defined. However, if the class has type parameters, any use of a self type must count as a use of those parameters. Failure to do so led to a soundness issue in Hack[6]:

Hack

```
// Counterexample by Derek Lam
class Base {}
class Derived extends Base {
    public function __construct(public int $derived_prop) {}
}
final class ImplCov<+T> {
    public function __construct(private T $v) {}
    public function put(this $v): void {
        $this->v = $v->v;
    }
    public function pull(): T {
        return $this->v;
    }
}
class Violate {
    public static function foo(ImplCov<Derived> $v): void {
        self::bar($v);
        echo $v->pull()->derived_prop;
        // Wait... Base doesn't have $derived_prop!
    }
    public static function bar(ImplCov<Base> $v): void {
        $v->put(new ImplCov(new Base()));
    }
}
// Violate::foo(new ImplCov(new Derived(42))); crashes
```

- **Local types** that are only used inside a single expression and do not escape do not need variance checking, because they don't form part of the interface. However, it's important to verify that they don't escape! Otherwise, a soundness bug arises, as in Scala[7]:

Scala

```
// Counterexample by Paul Phillips
class A[+T] {
  val foo0 = {
    class AsVariantAsIWantToBe { def contains(x: T) = () }
```

[4] Open GADTs and Declaration-site Variance: A Problem Statement, Paolo G. Giarrusso (2013)
[5] GADTs meet subtyping (ESOP '13) Gabriel Scherer and Didier Rémy (2013)
[6] github.com/facebook/hhvm/issues/7254 (2016)
[7] github.com/scala/bug/issues/5060 (2011)

```
    new AsVariantAsIWantToBe
  }
}

object Test {
  def main(args: Array[String]): Unit = {
    val xs: A[String] = new A[String]
    println(xs.foo0 contains "abc")
    println((xs: A[Any]).foo0 contains 5) // crashes
  }
}
```

- **Subclasses** may introduce new methods which do not respect covariance, producing an invariant subclass of a covariant class. This is not by itself unsound, but can cause unsoundness if the language also supports downcasting by pattern-matching, allowing covariant use of the invariant derived class through the covariant base class. This problem arose in Kotlin[8] and in Scala[9][10]:

Kotlin

```
// Counterexample by Ilya Gorbunov
// List is covariant, MutableList is an invariant subclass
private fun <E> List<E>.addAnything(element: E) {
    if (this is MutableList<E>) {
        this.add(element)
    }
}

arrayListOf(1, 2).addAnything("string")
```

Scala

```
// Counterexample by Chung-Kil Hur
sealed abstract class MyADT[+A]
case object MyNone extends MyADT[Nothing]
case class MyFun[A](fun: A=>A) extends MyADT[A]

def data : MyADT[Any] = MyFun((x:Int)=>x+1)

val foo : Any =
  data match {
    case MyFun(f) => f("a")
    case _ => 0
  }
```

[8] youtrack.jetbrains.com/issue/KT-7972 (2015)
[9] github.com/scala/bug/issues/8737#issuecomment-292432742 (2016)
[10] github.com/scala/bug/issues/6944 (2013)

4 Objects under construction

Mutation

There are two common styles for creating a new object or record. In the first style, common in functional languages, the programmer creates a new record by specifying the value of all of its fields in a single expression:

OCaml

```
type t = { foo : int; bar : string }
let make () = { foo = 42; bar = "hello" }
```

In the second style, common in imperative and object-oriented languages, the programmer creates a new record by first allocating a blank record, and then filling its fields in one-by-one using mutation:

C

```
struct t { int foo; const char* bar; };
struct t* x = malloc(sizeof(struct t));
x->foo = 42;
x->bar = "hello";
```

Java

```
class T {
  int foo;
  String bar;
  T () { this.foo = 42; this.bar = "hello"; }
}
```

In the first style, the record is fully constructed as soon as it is available, and no intermediate states are visible. By contrast, the second style allows intermediate states to be observed, which can have confusing or unsound consequences.

For instance, Java allows fields to be marked `final`, which disallows non-constructor assignments to the field, as in the following class:

Java

```
class A {
  private final int x;
  public void printX() { System.out.println(x); }
  public A () { printX(); this.x = 42; printX(); }
}
```

A Java programmer might believe that the `printX` method will always print the same value, but this is not the case as it may be invoked while the object is still under construction.

This caused a soundness issue in Kotlin's non-null checking[1], when a non-nullable field was observed to be null during initialisation.

Kotlin

```kotlin
// Counterexample by Joshua Rosen
class Foo {
    val nonNull: String

    init {
        this.crash()
        nonNull = "Initialized"
    }

    fun crash() {
        nonNull.startsWith("foo") // crashes
    }
}

fun main(args: Array<String>) {
    Foo()
}
```

In Java, it's additionally possible for `final` fields to never be initialised, because a reference to an uninitialised object can leak out via an exception:

Java

```java
class Ex extends RuntimeException {
    public Object o;
    public Ex(Object a) { this.o = a; }
    static int leak(Object x) { throw new Ex(x); }
}

class A {
    public final int leak = Ex.leak(this);
    public final int thing = Integer.parseInt("42");
}

public class Test {
    static A make() {
        try {
            return new A();
        } catch (Ex x) {
            return (A)x.o;
        }
    }
    public static void main(String []args){
       A a = make();
       // a is uninitialised: its 'thing' field is zero
       System.out.println(a.thing);
    }
```

[1] youtrack.jetbrains.com/issue/KT-10455 (2015)

```
}
```

5 Curry's paradox

Recursive types • Totality

In the simply-typed lambda calculus (which has only function types and a base type), infinite loops are impossible and all programs halt. Surprisingly, this stops being true once recursive types are added, even if no recursive functions or loops are present in the language.

In most languages, there are plenty of ways to write programs that do not terminate, and finding one more is not a soundness issue. However, in *total* languages (ones in which all programs halt), this does present a soundness issue and the allowable forms of recursive types must be restricted.

The recursive types in question are those that contain a function or method which takes the same type as an argument, which can be used to build a nonterminating computation as follows:

Haskell

```haskell
newtype Curry = Curry { r :: Curry -> Int }

f c = r c c
loop = f (Curry f)
```

OCaml

```ocaml
type curry = { r : curry -> int }

let f c = c.r c
let loop = f { r = f }
```

Java

```java
interface Curry {
  public int r(Curry x);
}
static int loop() {
  Curry c = new Curry() {
    public int r(Curry x) { return x.r(x); }
  };
  return c.r(c);
}
```

In logic, this is known as Curry's paradox.[1][2] (This has nothing to do with "function currying", other than being named after the same person.) Under the propositions-as-types viewpoint, it causes inconsistency: by replacing `Int` with any proposition P (including `False`), the `loop` function above proves P.

[1] en.wikipedia.org/wiki/Curry%27s_paradox
[2] plato.stanford.edu/entries/curry-paradox

To avoid this problem, languages that aim for logical consistency (e.g. the proof assistants Coq and Agda) ban recursive types that take themselves as arguments to functions or methods (so-called "negative recursion"), avoiding Curry's paradox.

(In fact, due to a different issue, banning negative recursion is often not enough, and recursive types must be restricted further to "strictly positive recursion" to remain consistent. See Positivity, strict and otherwise (§30))

6 Eventually, nothing

Empty types

Many type systems allow an empty type, which has no values:

Rust

```rust
enum Empty {}
```

OCaml

```ocaml
type empty = |
```

Haskell

```haskell
data Empty
```

The eliminator for the empty type allows it to be turned into a value of any other type, by handling each of the zero possible cases:

Rust

```rust
fn elim<T>(v : Empty) -> T { match v {} }
```

OCaml

```ocaml
let elim (x : empty) = match x with _ -> .
```

Haskell

```haskell
{-# LANGUAGE EmptyCase #-}
elim :: Empty -> a
elim x = case x of {}
```

The `elim` function claims to be able to produce a valid value of an arbitrary type, but will never need to actually do this since its input cannot be supplied.

In non-total languages is it possible to write an expression of type `empty`, by writing one that does not evaluate to a value but instead fails or diverges:

Rust

```rust
fn bottom() -> Empty { loop {} }
```

OCaml

```ocaml
let bottom : empty =
  let rec loop () = loop () in loop ()
```

Haskell

```haskell
bottom :: Empty
bottom = bottom
```

This interacts badly with an optimisation present in C compilers: according to the C standard, compilers are allowed to assume that the program contains no infinite loops without side-effects. (The point of this odd assumption is to allow the compiler to delete a loop that computes a value which is not used, without needing to first prove that the loop terminates).

So, infinite loops may be deleted when compiling using a backend designed for C and C++ (e.g. LLVM), allowing "values" of the empty type to be constructed:

Rust

```rust
let s : &str = elim(bottom());
println!("string: {}", s);
```

OCaml

```ocaml
(* The OCaml compiler does not delete empty loops, so
   computing bottom correctly hangs rather than crashing *)
print_endline (elim bottom)
```

Haskell

```haskell
-- GHC does not delete empty loops, so
-- this correctly hangs rather than crashing
putStrLn (elim bottom)
```

When optimisations are turned on, this program crashed in several versions of Rust[1].

[1] github.com/rust-lang/rust/issues/28728 (2015)

7 Dubious evidence

Empty types › Equality

While the `Empty` type of the previous counterexample has no values at all, there are types whose emptiness depends on their parameters. The canonical example is the equality type, expressible as a GADT in Haskell or OCaml or as an inductive family in Coq or Agda (among others):

OCaml

```
type (_, _) eq =
  Refl : ('a, 'a) eq
```

Haskell

```
data Eq a b where
  Refl :: Eq a a
```

Coq

```
Inductive eq {S : Set} : S -> S -> Prop :=
  Refl a : eq a a.
```

Agda

```
data Eq {S : Set} : S -> S -> Set where
  refl : (a : S) -> Eq a a
```

The type `(a, b) eq` witnesses equality: it is nonempty if `a` and `b` are the same type, and empty if they are distinct. Values of type `(a, b) eq` constitute evidence that `a` and `b` are in fact the same, and can be used e.g. to turn an `a` into a `b`.

For an arbitrary type `a`, we have no way of making an `(int, a) eq`: for all we know `a` might be `string`. The only way to construct a value of the `eq` type is using `Refl`, which demands the parameters be equal.

In a pure, total language, the existence of an expression of type `(a, b) eq` is enough to conclude `a` and `b` are equal. But in a less pure language, where evaluation might fail or loop, the mere existence of an expression with the right type is not evidence enough: we need to actually run it, to see that it does in fact yield `Refl`. Several type systems have had soundness bugs where evidence of this sort is trusted without being fully evaluated:

- **Null evidence**

  In languages with `null`, types like `eq` contain `null`, and unlike `Refl`, `null` implies nothing about its type parameters. So, before making use of any evidence we need to verify that the evidence isn't `null`.

The lack of such verification caused a soundness issue in Java and Scala[1] (using a type that provides evidence for subtyping, rather than equality):

Java

```java
// Counterexample by Nada Amin and Ross Tate
class Unsound {
  static class Constrain<A, B extends A> {}
  static class Bind<A> {
    <B extends A>
    A upcast(Constrain<A,B> constrain, B b) {
      return b;
    }
  }
  static <T,U> U coerce(T t) {
    Constrain<U,? super T> constrain = null;
    Bind<U> bind = new Bind<U>();
    return bind.upcast(constrain, t);
  }
  public static void main(String[] args) {
    String zero = Unsound.<Integer,String>coerce(0);
  }
}
```

Scala

```scala
// Counterexample by Nada Amin and Ross Tate
object unsoundMini {
  trait A { type L >: Any}
  def upcast(a: A, x: Any): a.L = x
  val p: A { type L <: Nothing } = null
  def coerce(x: Any): Nothing = upcast(p, x)
  coerce("Uh oh!")
}
```

- **Self-justifying evidence**

  Evidence `p : (int, a) eq` can be used to construct more evidence that `int` and `a` are equal, by seeing that `p` is `Refl`, therefore learning that `int` and `a` are equal, and using this information to justify `Refl : (int, a) eq`.

  OCaml[2] had a soundness issue in which it allowed this sort of reasoning in a recursive definition of `p`, allowing `p` to be used as evidence for itself:

  OCaml

  ```ocaml
  (* Counterexample by Stephen Dolan *)
  type (_,_) eq = Refl : ('a, 'a) eq
  let cast (type a) (type b) (Refl : (a, b) eq) (x : a) = (x : b)
  ```

[1] Java and Scala's Type Systems are Unsound (OOPSLA '16) Nada Amin and Ross Tate (2016)
[2] github.com/ocaml/ocaml/issues/7215 (2016)

```
let is_int (type a) =
  let rec (p : (int, a) eq) = match p with Refl -> Refl in
  p

let bang =
  (* segfaults *)
  print_string (cast (is_int : (int, string) eq) 42)
```

- **Out-of-order evidence**

  When nontermination is possible, it is important to ensure that the order in which evidence is used matches the evaluation order. Otherwise, it is possible to write an expression that does not terminate, but make use of it before it runs. Such forward references caused a soundness issue in Scala[3]:

  Scala

  ```
  // Counterexample by Paolo G. Giarrusso
  new {
    val a: String = (((1: Any): b.A): Nothing): String
    def loop(): Nothing = loop()
    val b: { type A >: Any <: Nothing } = loop()
  }
  ```

- **Time-travelling evidence**

  Staged metaprogramming allows a program to manipulate program fragments, gluing together an output program from quoted fragments of code. However, it is unsound to allow evidence computed in the future by the generated output program to justify computations now. This caused a soundness bug in Scala's implementation of staged metaprogramming[4] and in BER MetaOCaml[5]:

  Scala

  ```
  // Counterexample by Lionel Parreaux
  import scala.quoted.staging._

  given Toolbox = Toolbox.make(getClass.getClassLoader)
  trait T { type A >: Any <: Nothing }

  withQuoteContext { '{ (x: T) => ${ 42: x.A } } }
  // crashes with java.lang.ClassCastException
  ```

  OCaml

  ```
  (* Counterexample by Jeremy Yallop *)
  type _ t = T : string t
  ```

[3] github.com/lampepfl/dotty/issues/5854 (2019)
[4] github.com/lampepfl/dotty/issues/9353 (2020)
[5] github.com/metaocaml/ber-metaocaml/blob/ber-n111/ber-metaocaml-111/test/tgadt.ml (2016)

```
let f : type a. a t option code -> a -> unit code =
  fun c x -> .< match .~c with
  | None -> ()
  | Some T -> .~(print_endline x; .<()>.) >.

let _ = f .< None >. 0
```

8 Some kinds of Anything

Subtyping • Polymorphism

In type systems with polymorphism and subtyping, there are four different meanings that the phrase "any type" could have:

- **Top type** $\top$

  The top type is a supertype of every type. Any value can be turned into a value of type $\top$, but given a value of type $\top$ there's not much you can do with it.

- **Bottom type** $\bot$

  The bottom type is a subtype of every type. A value of type $\bot$ can be turned into a value of any type, but there's no way to construct a value of type $\bot$. (If there is, the type system is broken. See Eventually, nothing (§6) for one example).

- **Universal type** $\forall\alpha.\ \ldots$

  A value of a universal type $\forall\alpha.\varphi(\alpha)$ can be used at type $\varphi(X)$, for any type X. To construct one, the value must typecheck with an unknown abstract type α.

- **Existential type** $\exists\alpha.\ \ldots$

  A value of an existential type $\exists\alpha.\varphi(\alpha)$ can be constructed from a value of type $\varphi(X)$, for any type X. To use one, the use must typecheck with an unknown abstract type α.

There are a couple of relationships between these types:

- $\bot = \forall\alpha.\alpha$

  $\bot$ is the type that can be used at any type, but cannot be constructed.

- $\top = \exists\alpha.\alpha$

  $\top$ is the type that can be constructed from any type, but cannot be used.

In some systems, these relationships extend to more complicated types as well, for instance with a covariant type $\mathsf{List}[A]$ of immutable lists with elements of type A:

- $\mathsf{List}[\bot] = \forall\alpha.\mathsf{List}[\alpha]$

  The elements of a single list l cannot simultaneously be of every possible type α. Such a list must be empty, which is the same thing as being a list of $\bot$.

- $\mathsf{List}[\top] = \exists\alpha.\mathsf{List}[\alpha]$

  If a list contains elements of an unknown type α, then there is nothing useful we can do with them. The situation is the same as if we had a list of $\top$.

The extent to which languages implement these rules varies. Sometimes[1], they are used to justify converting $List[\bot]$ to $\forall\alpha.List[\alpha]$. Sometimes[2], they are used to justify considering those two as different spellings of the same type. Other times, they are not used, but are definable in the language.

However, if implementing these rules it is crucial to keep track of variance. Consider the type $\exists\alpha.(\mathsf{List}[\alpha] \to \mathsf{Int})$: this is a function that accepts lists of some particular unknown type. It could be, for instance, the sum function of type $\mathsf{List}[\mathsf{Int}] \to \mathsf{Int}$.

It is unsound to treat this the same as $\mathsf{List}[\top] \to \mathsf{Int}$, as that would allow us to pass a list of, say, strings to the sum function. In fact, since all of the occurrences of α are contravariant (that is, to the left of a single function arrow), this type is equivalent to $\mathsf{List}[\bot] \to \mathsf{Int}$.

Scala supports subtyping and polymorphism, and several versions of the language[3][4] had this soundness issue, where $\exists\alpha.\varphi(\alpha)$ was converted to $\varphi(\top)$ (spelled $\varphi(\mathsf{Any})$ in Scala), without regard for the variance of φ.

Scala

```scala
// Counterexample by Paul Chiusano
/* The 'coinductive' view of streams, represented as their unfold. */
trait Stream[+A]
case class Unfold[S,+A](s: S, f: S => Option[(A,S)]) extends Stream[A]

object Stream {
  def fromList[A](a: List[A]): Stream[A] =
    Unfold(a, (l:List[A]) => l.headOption.map((_,l.tail)))
}

/* If I have a Stream[Int], and I match and obtain an Unfold(s, f),
   the type of (s,f) should be (S, S => Option[(A,S)]) forSome { type S }.
   But Scala just promotes everything to Any: */
val res0 = Stream.fromList(List(1,2,3,4))
val res1 = res0 match { case Unfold(s, f) => s }
// res1: Any = List(1, 2, 3, 4)

/* Notice that the type of s is Any.
   Likewise, the type of f is also wrong, it accepts an Any: */

val res2 = res0 match { case Unfold(s, f) => f }
// res2: Any => Option[(Int, Any)] = <function1>

/* Since f expects Any, we can give it a String and get a runtime error: */

res0 match { case Unfold(s, f) => f("a string!") } // crashes
```

[1] Relaxing the Value Restriction, Jacques Garrigue (2004)
[2] Polymorphism, subtyping, and type inference in MLsub, Stephen Dolan and Alan Mycroft (2017)
[3] github.com/scala/bug/issues/5189 (2011)
[4] github.com/scala/bug/issues/6680 (2012)

9 Any (single) thing

Polymorphism

When typechecking a language with polymorphism (generics), it is usually up to the compiler to fill in the type parameters when a polymorphic function / generic method is called. For instance, given polymorphic function `id` of type $\forall\alpha.\alpha \to \alpha$, it can equally be called as `id(5)` and `id("hello")`, and the compiler will work out the correct value of α in both cases.

However, for any single call to `id` there is only one α to choose, and the compiler must choose consistently. The expression `id(“hello”) + 5` is a type error, even though instantiating α to `string` and `int` are both fine.

In practice, this means that whenever the compiler chooses a type parameter it must record that choice and check compatibility with any other use of the parameter. Early versions of Generic Java[1] and certain versions of Scala[2] and Kotlin[3] failed to do this in all cases, leading to unsoundness. The problem in each case was incorrectly allowing conversions from a type with one arbitrary parameter to a type with two arbitrary parameters, losing the constraint that the two parameters must be chosen the same way.

Java

```
// Counterexample by Alan Jeffrey
public class Problem {
    // This code compiles, but produces a ClassCastException when executed
    // even though there are no explicit casts in the program.

    public static void main (String [] args) {
        Integer x = new Integer (5);
        String y = castit (x);
        System.out.println (y);
    }

    static <A,B> A castit (B x) {
        // This method casts any type to any other type.
        // Oh dear.  This shouldn't type check, but does
        // because build () returns a type Ref<*>
        // which is a subtype of RWRef<A,B>.
        final RWRef<A,B> r = build ();
        r.set (x);
        return r.get ();
    }

    static <A> Ref<A> build () {
        return new Ref<A> ();
    }
}

interface RWRef<A,B> {
    public A get ();
```

[1] Generic Java type inference is unsound (TYPES mailing list), Alan Jeffrey (2001)
[2] github.com/scala/bug/issues/9410 (2015)
[3] youtrack.jetbrains.com/issue/KT-35679 (2019)

```
    public void set (B x);
}

class Ref<A> implements RWRef <A,A> {
    A contents;
    public void set (A x) { contents = x; }
    public A get () { return contents; }
}
```

Scala

```
// Counterexample by Stephen Compall
trait Magic {
  type S
  def init: S
  def step(s: S): String
}

case class Pair[A](left: A, right: A)

object Main extends App {
  type Aux[A] = Magic {type S = A}

  // I can't call this one from outerd,
  def outertp[A](p: Pair[Aux[A]]) =
    p.right.step(p.left.init)

  // but I can call this one.
  def outere(p: Pair[Aux[E]] forSome {type E}) =
    outertp(p)

  // This one means the left and the right may have *different* S.  I
  // shouldn't be able to call outere with p.
  def outerd(p: Pair[Magic]) =
    outere(p)

  def boom =
    outerd(Pair(new Magic {
                  type S = String
                  def init = "hi"
                  def step(s: S) = s.reverse
                },
                new Magic {
                  type S = Int
                  def init = 42
                  def step(s: S) = (s - 3).toString
                }))
  boom
}
```

Kotlin

```kotlin
// Counterexample by Victor Petukhov
class A<R1, K1>(val r1: R1, var k1: K1)
class B<R2, K2>(val r2: R2, val k2: K2)
interface I
class Q1 : I
class Q2 : I {
    fun x() = ""
}
fun <L, M> foo(x: A<L, M>, y: B<L, M>) {
    x.k1 = y.k2
}
inline fun <reified T : I> bar(): B<T, T> {
    val w = T::class.constructors.first().call()
    return B(w, w)
}
fun main() {
    val a1 = A(Q1(), Q2())
    val a = a1 as A<Q1, out Any?>
    foo(a, bar()) // type mismatch in NI
    println(a1.k1.x())
}
```

10 Mutable matching

Mutation

Languages with sum types / algebraic datatypes generally include some form of pattern-matching: a `match`/`case`/`switch` construct that can inspect values.

The cases in a match statement often contain redundancy, and compilers optimise these statements by coalescing repeated subpatterns. For instance, consider this OCaml match:

OCaml

```
match x, y with
| 42, 0 -> "foo"
| 42, n -> "bar"
| n, _ -> "baz"
```

The compiled code will first check whether $x = 42$, and if so then checks whether $y = 0$. The optimisation is that if $y \neq 0$, the code skips straight to the `"bar"` outcome, without rechecking whether $x = 42$.

The assumption here is that x did not change between the first pattern and the second. This assumption can be violated by the presence of two features: first, the ability to pattern-match on mutable fields, and second, the ability to run arbitrary code during pattern matching. (The second feature usually shows up under the name "guards": additional arbitrary conditions that are checked before the match succeeds).

If the language contains existential types (e.g. GADTs in OCaml, abstract type members in Scala), then it is possible for x to not only change value but change type between two patterns, which leads to unsoundness. This issue appeared in both OCaml[1] and Scala[2], which both support mutable matches, arbitrary conditions in guards, and existential types. Rust[3] also had the issue, with the unsoundness there resulting from disagreement about lifetime rather than type.

OCaml

```
(* Counterexample by Stephen Dolan *)
type app = App : ('x -> unit) option * 'x -> app

let app1 = App (Some print_string, "hello")
let app2 = App (None, 42)

type t = {
  a : bool;
  mutable b : app
}

let f = function
| { a = false } -> assert false
```

[1] github.com/ocaml/ocaml/issues/7241 (2016)
[2] github.com/scala/bug/issues/6070 (2012)
[3] github.com/rust-lang/rust/issues/27282 (2015)

```
| { a = true; b = App (None, _) } -> assert false
| { a = true; b = App (Some _, _) } as r
    when (r.b <- app2; false) -> assert false
| { b = App (Some f, x) } ->
   f x

let _ = f { a = true; b = app1 }
```

Scala

```
// Counterexample by Iulian Dragos
abstract class Bomb {
    type T
    val x: T

    def size(that: T): Int
}

class StringBomb extends Bomb {
    type T = String
    val x = "abc"
    def size(that: String): Int = that.length
}

class IntBomb extends Bomb {
    type T = Int
    val x = 10

    def size(that: Int) = x + that
}

case class Mean(var bomb: Bomb)

object Main extends App {
    def foo(x: Mean) = x match {
        case Mean(b) =>
            // b is assumed to be a stable identifier,
            // but it can actually be mutated
            println(b.size({ mutate(); b.x }))
    }

    def mutate() {
        m.bomb = new IntBomb
    }

    val m = Mean(new StringBomb)
    foo(m)
}
```

Rust

```rust
// Counterexample by Ariel Ben-Yehuda
fn main() {
    match Some(&4) {
        None => {},
        ref mut foo
            if {
                (|| { let bar = foo; bar.take() })();
                false
            } => {},
        Some(s) => println!("{}", *s)
    }
}
```

This can be fixed by disallowing pattern-matching on mutable fields, or disallowing mutation during guards. (Either suffices). A trickier solution chosen by both OCaml and Scala is to write the pattern-match compiler extremely carefully so that it never loads a mutable field twice, and so never assumes two possibly-distinct values to be equal.

11 Runtime type misinformation

Typecase

The typing judgement $e : A$ is a static, syntactic judgement based on the syntax of e and A. Sometimes, it would be useful to have a runtime counterpart, allowing expressions like $e \ ? \ A$ which evaluate to `true` if at runtime the expression e evaluates to a value of type A. This "?" operator goes by various names, including `typecase` and `instanceof`.

In any but the simplest type systems, this is fraught with difficulties. The trouble is that advanced type systems can make finer distinctions than are visible at runtime, so the runtime type check has incomplete information.

For example, using `newtype` in Haskell it is possible to define a type `EscapedString` as a new type based on `String`:

Haskell

```
newtype EscapedString = EscapedString String
```

Crucially, unlike type aliases (`type` in Haskell, or `typedef` in C/C++) the two types `String` and `EscapedString` are distinct for type checking purposes, causing a type error whenever one is used where the other is expected and ensuring that a forgotten escaping or unescaping causes a compile failure.

This extra checking has no runtime cost, because `newtype`s in Haskell (and opaque types in ML, and similar features) introduce no extra wrappers: the `String` and `EscapedString` have the same representation.

This is fundamentally incompatible with typecase. The expression `e ? EscapedString` cannot be expected to return `true` on escaped strings and `false` on unescaped ones, because at runtime there is no distinction between them. The situation gets even worse with more advanced type systems, since whether e has type A may not even be decidable at runtime.

It is possible to mix typecase with advanced type systems by limiting the typecase operator to those types for which sufficient runtime information is available, either by distinguishing static types from dynamic tags (e.g. ML's `exn`, OCaml's open types), or by specifying the subset of static types that have dynamically-checkable representations (e.g. Haskell's `Typeable`, Scala's "checkable types").

When this is not done carefully, unsoundness results, in which a runtime tests identifies two types that are statically known to be distinct. This problem has occurred in Scala[1] (where the types checked in patterns did not always match the static ones) and some cases remain in Dotty[2]. The problem also occurs in Hack[3] (where information about generics was erased at runtime, so typecase returned `true` if asked whether a list of ints was a `List<string>`), in Flow[4] (which also performs typecase on erased generics), and in Kotlin[5] (where inner classes share runtime information, even if they depend on generic parameters that may vary).

[1] github.com/scala/bug/issues/1503 (2008)
[2] github.com/lampepfl/dotty/issues/9359 (2020)
[3] github.com/facebook/hhvm/pull/7632 (2017)
[4] github.com/facebook/flow/issues/6741 (2018)
[5] youtrack.jetbrains.com/issue/KT-9584 (2015)

Java has an issue similar to those in Hack and Kotlin, where `instanceof` checks ignore generic parameters, but avoids unsoundness by limiting which classes can be used for comparison.

Scala

```scala
// Counterexample by David R. MacIver
object Test extends Application {
    case class L();
    object N extends L();

    def empty(xs : L) : Unit = xs match {
        case x@N => println(x); println(x);
    }

    empty(L())
}
/*
The compiler inserts a cast of xs to N.type, which is unsound:
The pattern match will succeed for any L, because N == L().
*/
```

Hack

```hack
// Counterexample by Andrew Kennedy
function mycast<T>(mixed $x, classname<T> $t) : T {
  if ($x instanceof $t) { return $x; }
  else throw new Exception("Type didn't match");
}
function expectListOfString(List<string> $x) { ... }

// If I write expectListOfString(mycast($y, List::class))
// but with $y having type List<int> then no exception will
// get thrown at runtime (because of generics erasure).
```

Flow

```flow
// Counterexample by William Chargin
// @flow
class Box<T> {
  +field: T;
  constructor(x) {
    this.field = x;
  }
}

function asBox<T>(x: T | Box<T>): Box<T> {
  if (x instanceof Box) {
    // Here, `x` is refined to be `Box<T>`. This is unsound: `T` could
    // well be `Box<U>` for some other `U`. Example below.
    return x;
  } else {
    // whatever
```

```
    return new Box(x);
  }
}

const stringStringBox: Box<Box<string>> = asBox(new Box("wat"));  // unsound!
stringStringBox.field.field.substr(0);  // runtime error
```

Kotlin

```
// Counterexample by Vladimir Reshetnikov
class A<T>(var value: T) {
    fun replaceValue(x: Any) : Any {
        class C(var v: T)
        if(x is C) {
            value = x.v
            return x
        }

        return C(value)
    }
}

fun main(args: Array<String>) {
    val a = A("string")
    a.replaceValue(A(0).replaceValue("something not of type C"))

    // a.value has type String, but now contains integer 0
    val s = a.value // crashes
}
```

Java

```
// This function would do an unsound conversion from Integer to String
// because the cls.cast always passes: it's only checking List
static String bad(Class<List<String>> cls) {
    List<Integer> i = Arrays.asList(42);
    List<String> badList = cls.cast(i);
    String badString = badList.get(0);
    return badString;
}

// But Java provides no way to get a Class<List<String>>.
// (Arrays.asList("a").getClass() is a Class<? extends List>)
```

Runtime type tests are also tricky when the value being tested has a union type: see Polymorphic union refinement (§29).

12 Overloading and polymorphism

Polymorphism • Overloading

Polymorphism allows a single method to work with an arbitrary, unknown type, while overloading allows one of multiple methods to be selected by examining the types of the parameters. With overloading, the parameter types become part of the name of the method. With polymorphism, the parameter types might not be known.

The two features are in direct conflict, because the information that overloading requires is unavailable in a polymorphic context. Attempts to combine them are tricky, as this counterexample in Java[1] shows:

Java

```java
// Counterexample by Hiromasa Kido
class A{
	public int compareTo(Object o){ return 0; }
}
class B extends A implements Comparable<B>{
	public int compareTo(B b){ return 0; }
	public static void main(String[] argv){
		System.out.println(new B().compareTo(new Object()));
	}
}
```

Scala

```scala
// Counterexample by Kota Mizushima
// (translation of Java counterexample, with the same effect)
class A {
  def compareTo(o: Any): Int = 0
}
class B extends A with Comparable[B] {
  def compareTo(b: B): Int = 0
}
object C {
  def main(args: Array[String]): Unit = {
    println(new B().compareTo(new Object()))
  }
}
```

On earlier versions of Java (and Scala[2]), this program crashed with a `ClassCastException`, despite containing no casts. The issue is that in order to implement `B`'s `compareTo(B)`, the compiler inserts a "bridge method" `compareTo(Object)` containing a cast to `B`. The bridge method is necessary because the `compareTo` method is specified by `Comparable`, and other users of the `Comparable` interface will select the `compareTo(Object)` overload, as they do not necessarily know about `B`. However, this bridge method accidentally overrides `A`'s `compareTo(Object)` method, and gets called from `main` instead, and the cast fails.

[1] Java generics unsoundness? (TYPES mailing list), Eijiro Sumii (2006)
[2] github.com/scala/bug/issues/9912 (2016)

In current Java, bridge methods still exist, but the program above is rejected.

Scala[3] additionally has a related issue, also caused by an interaction of overloading and polymorphism. Scala allows a mixture of structural and nominal typing. An object can be given a structural type that exposes only some of its capabilities, including exposing only special cases of polymorphic methods:

Scala

```
// Counterexample by Paul Phillips
object Test {
  class MyGraph[V <: Any] {
    def addVertex(v: V): Boolean = true
  }

  type DuckGraph = {
    def addVertex(vertex: Int): Boolean
  }

  def fail(graph: DuckGraph) = graph addVertex 1

  def main(args: Array[String]): Unit = {
    fail(new MyGraph[Int])
  }
}
```

However, overloading causes this program to fail, by attempting to find a nonexistent `addVertex(Int)` method, even though the underlying polymorphic method has signature `addVertex(Object)`.

[3] github.com/scala/bug/issues/2672 (2009)

13 Distinctness I: Injectivity

Injectivity

Type checkers spend much of their time verifying that pairs of types are equal, or at least compatible, checking for instance that the type of the argument to a function matches the type it expects.

Occasionally, they need to do the opposite, and verify that two types are distinct. One common case of this is allowing the programmer to omit those cases of a pattern match which are impossible because they would imply that two distinct types are equal. For instance, given a value which is either a string, or evidence that Int = String, then matching on this value need only handle the first case as the second is impossible.

OCaml

```
type (_,_) eq = Refl : ('a, 'a) eq
type ('a, 'b) either = Left of 'a | Right of 'b

let f (x : (string, (int,string) eq) either) =
  match x with
  | Left s -> s
  (* no case for Right, and yet the match is exhaustive *)
```

Agda

```
-- Agda can reason about distinctness of more than just types
open import Agda.Builtin.String
data Eq (a : String) : String -> Set where
  Refl : Eq a a

data Sum (A B : Set) : Set where
  Left : A -> Sum A B
  Right : B -> Sum A B

f : Sum String (Eq "foo" "bar") -> String
f (Left s) = s
f (Right ()) -- "()" is the absurd pattern
```

Of course, this is unsound if the type system is wrong about two types being distinct. One common way that this can occur is when the type system assumes that all type constructors are *injective*.

Injectivity of a type constructor F means that if $F[A] = F[B]$, then the types A and B are equal. Most common type constructors (`List`, `Array` and so on) are in fact injective, so this is a natural property to expect. However, if it is possible to define a type constructor F by $F[A] = \mathsf{Int}$, then injectivity fails. This issue occurred in Dotty[1], a research Scala compiler, as well as in Agda[2] (with the `injective-type-constructors` option). Curiously, Haskell makes the same assumption, but without unsoundness.

[1] github.com/lampepfl/dotty/issues/5658 (2018)
[2] github.com/agda/agda/issues/2250 (2016)

Scala

```scala
// Counterexample by Aleksander Boruch-Gruszecki
object Test {
  sealed trait EQ[A, B]
  final case class Refl[T]() extends EQ[T, T]

  def absurd[F[_], X, Y](eq: EQ[F[X], F[Y]], x: X): Y = eq match {
    case Refl() => x
  }

  var ex: Exception = _
  try {
    type Unsoundness[X] = Int
    val s: String = absurd[Unsoundness, Int, String](Refl(), 0)
  } catch {
    case e: ClassCastException => ex = e
  }

  def main(args: Array[String]) =
    assert(ex != null)
}
```

Agda

```agda
-- Counterexample by Andreas Abel
{-# OPTIONS --injective-type-constructors #-}

open import Common.Prelude
open import Common.Equality

abstract
  f : Bool → Bool
  f x = true

  same : f true ≡ f false
  same = refl

not-same : f true ≡ f false → ⊥
not-same ()

absurd : ⊥
absurd = not-same same
```

Haskell

```haskell
data Equ a b where
  Refl : Equ a a

-- not in fact unsound! see below
everythingIsInjective :: Equ (f a) (f b) -> Equ a b
everythingIsInjective Refl = Refl
```

The reason that it is sound for Haskell to assume an arbitrary type constructor F is injective is that it draws a distinction between type constructors and type-level functions. The following two declarations are distinct:

Haskell

```haskell
type family Foo :: * -> *
type family Bar a :: *
```

Here, `Foo` is defined as a type constructor, something of kind `* -> *`. This kind only contains injective type constructors (like lists, etc.). By contrast, `Bar` is defined as a type-level function. `Bar` is not necessarily injective, as one can define:

Haskell

```haskell
type instance Bar a = Int
```

However, `Bar` is not of kind `* -> *`, and cannot be passed as the `f` parameter to `everythingIsInjective`:

Haskell

```haskell
eqFoo :: Equ (Foo a) (Foo b) -> Equ a b
eqFoo = everythingIsInjective -- works

eqBar :: Equ (Bar a) (Bar b) -> Equ a b
eqBar = everythingIsInjective -- error
```

In fact, in Haskell there is currently no way to quantify over things like `Bar`: Haskell's higher-kinded types only allow passing around injective type constructors. A recent paper by Kiss et al.[3] proposes adding parameterisation by type functions to Haskell, by adding a new kind `* => *` which does not carry injectiveness assumptions.

[3] Higher-order Type-level Programming in Haskell (ICFP '19), Csongor Kiss, Susan Eisenbach, Tony Field, Simon Peyton Jones (2019)

14 Distinctness II: Recursion

Recursive types

(This is the same sort of problem as the previous counterexample (§13), so read that first for context.)

When checking distinctness of two types that contain type variables, one approach is to attempt to unify them and see whether there is a common unifier. (Be careful! It is easy to assume injectivity (§13) this way).

Unification can fail because of the *occurs check*, under which a type variable cannot be unified with a type mentioning the same variable. For instance, α and $\alpha \to \alpha$ fail to unify because of this check.

So, if using unification to check distinctness, it is natural to believe that α and $\alpha \to \alpha$ are distinct, regardless of what α is. However, this is only sound if there are no infinite or recursive types in the language: if a type T could be constructed equal to $T \to T$, then α and $\alpha \to \alpha$ would no longer be distinct.

Even when they are not directly supported, it is easy for infinite or recursive types to sneak into the language, which creates a soundness issue in languages checking distinctness by unification. This occurred in both Haskell[1] and OCaml[2], for the same reason: if the definition of a recursive type is split across module boundaries (multiple files with type families in Haskell, or a single recursive module in OCaml), then the typechecker will never see the construction of the whole recursive type and so cannot reject it. This allows a counterexample to distinctness, which can be exploited either via a Haskell type family or and OCaml GADT match.

Haskell

```haskell
-- Counterexample by Akio Takano
-- Base.hs
{-# LANGUAGE TypeFamilies #-}
module Base where

-- This program demonstrates how Int can be cast to (IO String)
-- using GHC 7.6.3.
type family F a
type instance F (a -> a) = Int
type instance F (a -> a -> a) = IO String

-- Given this type family F, it is sufficient to prove
-- (LA -> LA) ~ (LA -> LA -> LA)
-- for some LA. This needs to be done in such a way that
-- GHC does not notice LA is an infinite type, otherwise
-- it will complain.
--
-- This can be done by using 2 auxiliary modules, each of which
-- provides a fragment of the proof using different partial knowledge
-- about the definition of LA.
```

[1] ghc.haskell.org/trac/ghc/ticket/8162 (2013)
[2] github.com/ocaml/ocaml/issues/6993 (2015)

```haskell
--
-- LA -> LA
-- = {LA~LB->LB} -- only Int_T.hs knows this
-- LA -> LB -> LB
-- = {LA~LB}     -- only T_IOString.hs knows this
-- LA -> LA -> LA
type family LA
type family LB
data T = T (F (LA -> LB -> LB))

-- Int_T.hs
{-# LANGUAGE TypeFamilies, UndecidableInstances #-}
module Int_T where
import Base
type instance LA = LB -> LB

int_t0 :: Int -> T
int_t0 = T

-- T_IOString.hs
{-# LANGUAGE TypeFamilies, UndecidableInstances #-}
module T_IOString where
import Base
type instance LB = LA

t_ioString :: T -> IO String
t_ioString (T x) = x

-- Main.hs
import Int_T
import T_IOString

main :: IO ()
main = t_ioString (int_t0 100) >>= print
```

OCaml

```ocaml
(* Counterexample by Stephen Dolan *)
type (_, _) eqp = Y : ('a, 'a) eqp | N : string -> ('a, 'b) eqp
let f : ('a list, 'a) eqp -> unit = function N s -> print_string s

(* Using recursive modules, we can construct a type t = t list,
   even without -rectypes: *)

module rec A : sig
  type t = B.t list
end = struct
  type t = B.t list
end and B : sig
  type t
  val eq : (B.t list, t) eqp
end = struct
  type t = A.t
```

```
  let eq = Y
end

(* The expression f B.eq segfaults *)
```

15 Distinctness III: Options

Many compilers for typed languages accept options which affect type checking, and to ensure soundness, some care must be taken if mixing files typechecked with different options.

Options which enable or disable new type system features are relatively unproblematic, but trouble can occur if options affect existing types: if two types are considered equal with the option but are provably distinct without it, then it is unsound to combine two files compiled with and without the option.

This has caused a soundness bug in Agda[1], by mixing code compiled with the `--cubical` option (which permits multiple proofs of equality) and the `--with-K` option (which assumes equalities have unique proofs). Similar bugs arose several times in OCaml, based on various options that affect type equality: `--rectypes`, which permits recursive types[2], `--safe-string`, which makes the immutable `string` and mutable `bytes` type distinct[3], and `--nolabels`, which coarsens equality of function types with named parameters[4].

OCaml

```
(* Counterexample by Gabriel Scherer and Benoît Vaugon *)
(* blah.ml, compiled without -rectypes *)
type ('a, 'b, 'c) eqtest =
  | Refl : ('a, 'a, float) eqtest
  | Diff : ('a, 'b, int) eqtest

let test : type a b . (unit -> a, a, b) eqtest -> b = function
  | Diff -> 42

(* bluh.ml, compiled with -rectypes *)
let () =
  print_float Blah.(test (Refl : (unit -> 'a as 'a, 'a, _) eqtest))
```

These sorts of issues are conspicuously absent from GHC Haskell, which supports an unusually large number of options that affect typechecking. A reason for this is that internally, GHC compiles Haskell with any set of extensions to the same typed intermediate language, Core. Since the Core language is unaffected by options, incompatibilities between options would cause one of them to generate ill-typed Core, which can be detected locally.

[1] github.com/agda/agda/issues/2487#issuecomment-444498567 (2017)
[2] github.com/ocaml/ocaml/issues/6405 (2014)
[3] github.com/ocaml/ocaml/issues/7113 (2016)
[4] github.com/ocaml/ocaml/issues/7432 (2016)

16 Subtyping vs. inheritance

Subtyping

In an object-oriented language, after writing a class `A` there are two different ways we might want to extend it:

- **Subtyping** means writing a class `B` which conforms to `A`'s interface, as well as possibly adding some new methods of its own. Per Liskov's substitution principle[1], in any context where an `A` is expected we can supply a `B` instead.

- **Inheritance** means writing a class `B` that specialises `A` to a particular use, by reusing some of its behaviour and perhaps overriding parts.

The two are similar: if the class `B` reuses all of `A`'s functionality and adds some new methods of its own, then `B` will both inherit from and be a subtype of `A`.

However, they are not the same. Suppose `B` inherits most of its behaviour from `A`, but overrides a single method `m`. If `B` is a specialised version of `A`, its `m` might require a specialised input, accepting only a specialised version of `A.m`'s input. On the other hand, if `B` is a subtype of `A`, then in order to conform `A`'s interface its `m` must accept *all* inputs that `A.m` accepts, requiring the input of `B.m` to be a *supertype* of that of `A.m`.

Many object-oriented languages conflate inheritance and subtyping as *subclassing*. Three representative examples are C#, Eiffel and TypeScript, which differ in how overridden methods in a subclass are typechecked.

C# insists that subclasses' methods accept exactly the same types the overridden method accepts, which is sound but makes some uses of subtyping and specialisation awkward. Eiffel prefers specialisation, insisting that subclasses' methods accept subtypes of what the overridden method accepts, which is unsound[2]. TypeScript is ambivalent, requiring only that subclasses' methods accept either a supertype *or* a subtype of what the overridden method accepts, which is also unsound.

The soundness issue is the same one in both TypeScript and Eiffel: Suppose a method `A.m` may be overridden by `B.m`, where `B.m` accepts only a subtype of the original type. Since `B` is deemed a subtype of `A`, a `B` can be used as though it were an `A`, and by invoking its `B.m` method through `A` an argument which is not of the subtype it expects can be supplied.

TypeScript

```
interface Base {
    name: string;
}
interface Sub {
    name: string;
    doStuff: () => number;
}

class A {
```

[1] A behavioral notion of subtyping, Barbara Liskov and Jeannette Wing (1994)
[2] A Proposal for Making Eiffel Type-safe, W. R. Cook (1989)

```
    go(_ : Base) {}
}
class B extends A {
    go(x : Sub) { x.doStuff(); }
}

let x : Base = { name: "x" };
let b = new B();
let a : A = b;
a.go(x); // crashes
```

Eiffel

```
-- Counterexample by W. R. Cook
class Base feature
  base (n : Integer) : Integer
    do Result := n * 2 end;
end
---
class Extra inherit Base feature
  extra (n : Integer) : Integer
    do Result := n * n end;
end
---
class P2 feature
  get (arg : Base) : Integer
    do Result := arg.base(1) end
end
---
class C2 inherit P2 redefine get end feature
  get (arg : Extra) : Integer
    do Result := arg.extra(2) end
end
---
  local
    a : Base
    v : P2
    b : C2
    i : Integer
  do
    create a;
    create b;
    v := b;
    i := v.get(a) -- crashes!
  end
```

OCaml

```
class type base = object
  method name : string
end
class type sub = object
```

```
  method name : string
  method do_stuff : int
end

class a = object
  method go (_ : base) = ()
end

class b = object (self : < go : sub -> int; .. >)
  (* does not compile *)
  inherit a
  method! go (x : sub) = x#doStuff
end
```

C#

```
class A {
  public void go(Object arg) {}
}
class B : A {
  // does not compile
  public override void go(String arg) {}
}
```

Since version 2.6, TypeScript supports the `strictFunctionTypes` option[3], which uses a stricter subtyping check in some cases. However, the counterexample above is still accepted with `strictFunctionTypes`.

Theoretically, Eiffel recovers soundness by the "system validity check", a whole-program dataflow analysis designed to detect situations like Cook's counterexample (which the Eiffel community terms "catcalls", for "Changed Availability or Type"). However, this check is quite tricky, and it appears that no Eiffel compilers have ever actually implemented it[4].

The same issue can crop up with *class methods*, which are methods that are associated with a class rather than with an instance of that class. Languages with class methods allow classes to be passed around as values, dispatching method calls to the appropriate class as determined at runtime.

When class methods can be overridden in a subclass, this raises the same subtyping vs. inheritance issue: may the subclass specialise its argument type, or must it accept a supertype? A soundness issue along these lines (analogous to the one above) arose in Swift[5]:

Swift

```
// Counterexample by Ben Pious
class C<T> {

    let t: T
```

[3] typescriptlang.org/docs/handbook/release-notes/typescript-2-6.html (2017)
[4] Catching CATs, Markus Keller (2003)
[5] bugs.swift.org/browse/SR-7573 (2018)

```
    init(t: T) {
        self.t = t
        type(of: self).f()(self)
    }

    class func f<U>() -> (U) -> () where U: C  {
        return { (u: U) in
            print(u.t)
        }
    }
}

class E {
    let g = "Expected to Print"
}

class D: C<E> {
    override class func f<U>() -> (U) -> () where U: E {
        return { (u: E) in
            print(u.g)
        }
    }
}

let d = D(t: E()) // prints random garbage
```

17 Selfishness

Subtyping

Most statically typed object-oriented languages allow a group of related methods to be specified together as an *interface* (or "trait", "protocol", etc.). *Self types* are a feature that allows the types in such an interface to refer to the type implementing that interface.

Using self types as method arguments allows precise typechecking of *binary methods*[1] such as `equals`, where `x.equals(y)` requires a `y` of the same type as `x`. The lack of such types in e.g. Java means that the `Object.equals` method must usually include a cast that can fail at runtime, as its type allows any other object to be passed.

Using self types as returns allows precise typechecking of methods that return `this`, or methods that copy the receiving object.

However, the presence of self types breaks some properties of the type system, and unsoundness arises if other parts of the system rely on them.

- Argument self types break the property that, if a class `C` has a subclass `D` and both implement an interface `P`, then a `D` can be used anywhere a `C` is expected. The issue is that `P` may include a method that accepts a self type, and this method in `D` works on a narrower class of inputs than the corresponding method in `C`. (See Subtyping vs. inheritance (§16))

OCaml

```
class c (name : string) =
  object (self : 'self)
    method name = name
    method equals (x : 'self) =
      (name = x#name)
  end

class d (name : string) (size : int) =
  object
    inherit c name as super
    method size = size
    method equals (x : 'self) =
      (super#equals x && size = x#size)
  end

let sub (x : d) = (x :> c)
(* OCaml correctly rejects this coercion:
   despite inheriting from it, d is not a subtype of c *)
```

- Returned self types break the property that, if a class `C` implements an interface `P`, and subclass `D` inherits all of its behaviour from `C`, then `D` also implements `P`.

  The issue is that `P` may include a method that returns `Self`, which `C` implements by returning a new `C`. When this method is inherited by `D` it still returns `C`, even though `P`

[1] On Binary Methods, Kim Bruce, Luca Cardelli, Giuseppe Castagna, The Hopkins Objects Group, Gary T. Leavens, and Benjamin Pierce (1995)

now requires that it return `D`. This led to a soundness issue in Swift[2], and a related issue in Rust[3]:

Swift

```swift
// Counterexample by Hamish Knight
protocol P {
  associatedtype X where X == Self
  func foo() -> X
}

class C : P {
  typealias X = C
  func foo() -> X {
    return C()
  }
}
class D : C {
    var name = "D"
}

func foo<T : P>(_ x: inout T) {
  x = x.foo()
}

var d = D()
foo(&d)
print(d.name) // crashes
```

Rust

```rust
// Counterexample by Niko Matsakis
trait Make {
    fn make() -> Self;
}

impl Make for *const uint {
    fn make() -> *const uint {
        ptr::null()
    }
}

fn maker<M:Make>() -> M {
    Make::make()
}

fn main() {
    let a: *uint = maker::<*uint>();
    // we have "produced" a *uint even though there is no
    // function in this program that returns one.
}
```

[2] bugs.swift.org/browse/SR-10713 (2019)
[3] github.com/rust-lang/rust/issues/5781 (2013)

An alternative to self types is to use generic interfaces: instead of an interface Equals with an `equals` method accepting a self type, one can use a generic interface Equals[A] whose `equals` method accepts an A, and then write classes C that implement Equals[C]. This is the approach taken by C#'s `IEquatable<T>` and Java's `Comparable<T>`.

18 Privacy violation

Subtyping • Variance

With declaration-site variance (see Covariant containers (§2)), a generic class can be declared to be covariant in its type parameter, as long as the type parameter is only used in output positions:

C#

```
class Box<+X> {
  // allowed, X is an output (covariant)
  public X get() { ... }

  // disallowed, X is an input (contravariant)
  public void set(X x) { ... }
}
```

Since variance checking is verifying subtyping, a property of the public interface to a class, it should in principle be fine to allow methods that use `X` contravariantly (like `set` above), provided that they are marked `private` and therefore do not form part of the interface.

Whether or not this is sound hinges on exactly what the `private` modifier means: is it private to the *object* or private to the *class*? Conventionally, many object-oriented languages choose class-private, allowing access to private fields of an object of class `A` from any method of class `A`, regardless of whether the object being accessed was `this` or any other.

Choosing class-private makes the private variance check unsound, as pointed out by Emir et al.[1] in their paper introducing declaration-site variance for C#. (The same issue later reappeared in Hack[2]).

C#

```
// Counterexample by Emir et al.
class Bad<+X> {
  private X item;
  public void BadAccess(Bad<string> bs) {
    Bad<object> bo = bs;
    bo.item = new Button(); // we just wrote a button as a string
  }
}
```

Hack

```
// Counterexample by Andrew Kennedy
class Box<+T> {
  // OK, we've got a private field whose type involves the covariant T
  public function __construct(private T $elem) {
  }
```

[1] Variance and Generalized Constraints for C# Generics (ECOOP '06), Burak Emir, Andrew Kennedy, Claudio Russo, Dachuan Yu (2006)

[2] github.com/facebook/hhvm/issues/7216#issuecomment-235726828 (2016)

```
  // As usual, a (safe) getter method
  public function get(): T { return $this->elem; }

  // Private gives us access to arbitrary instances of Box, even in static
  // methods. Note the use of covariant subtyping to put a string in
  // a Box<mixed>
  public static function updateAsString(Box<mixed> $x, string $s) : void {
    $x->elem = $s;
  }

  // We can now use this method to overwrite an integer with a string
  // but return it as an integer
  public static function morphIntToString(int $i) : int {
    $x = new Box($i);
    Box::updateAsString($x, 'hey you turned me into a string');
    return $x->get();
  }

  // Actually do it
  public static function useBox(): void {
    $i = Box::morphIntToString(23);
    echo('this should be an integer: ' . $i);
  }
}
```

Scala supports both `private` (class-private) and `private[this]` (object-private), with different variance checking. It also supports `protected[this]`, an "object-protected" qualifier in which a field is accessible only via `this`, but both by the class itself and its subclasses.

Scala's `protected[this]` has the same variance-checking as `private[this]`, allowing non-covariant uses of a type parameter in a `protected[this]` field, even from within a class marked as covariant. This is an extraordinarily subtle feature, and Scala's current implementation has a number of soundness issues.

First, Scala supports a form of multiple inheritance via "traits". A class can inherit from a covariant trait in multiple ways, and the two copies of the trait's interface are merged. This merging can be justified by covariance, but `protected[this]` allows classes to provide non-covariant features to subclasses. This is unsound, as different traits in the inheritance heirarchy can see different values of the type parameter in non-covariant ways[3]:

Scala

```
// Counterexample by Jason Zaugg
trait A[+X] {
  protected[this] def f(x: X): X = x
}

trait B extends A[B] {
  def kaboom = f(new B {})
}
```

[3] github.com/scala/bug/issues/7093 (2013)

```
// protected[this] disables variance checking
// of the signature of `f`.
//
// C's parent list unifies A[B] with A[C]
//
// The protected[this] loophole is widely used
// in the collections, every newBuilder method
// would fail variance checking otherwise.
class C extends B with A[C] {
  override protected[this] def f(c: C) = c
}

// java.lang.ClassCastException: B$$anon$1 cannot be cast to C
//  at C.f(<console>:15)
new C().kaboom
```

Second, Scala allows `protected[this]` to apply also to abstract type members, which can be exposed by a subclass[4]:

Scala

```
// Counterexample by Derek Lam
abstract class Base[+T] {
  protected[this] type TSeq <: MutableList[T]
  val v: TSeq
}

class Sub extends Base[Int] {
  type TSeq = MutableList[Int]
  val v = MutableList(42)
}

val x = new Sub()
(x: Base[Any]).v += "string!"
x.v.last + 42
// java.lang.ClassCastException: java.lang.String cannot be cast to java.lang.Integer
```

[4] github.com/scala/scala-dev/issues/370 (2017)

19 Unstable type expressions

Mutation

Types can be passed around as first-class values in several languages, either directly (particularly in languages with dependent types and universes) or as part of another structure (e.g. first-class modules in OCaml, abstract type members in Scala).

This means that it is possible to have a type expression T that depends on a value: if some boolean b is false, then $T = \mathsf{Int}$, but if it's true then $T = \mathsf{String}$. In languages with mutation, there can even be a type expression T where $T \neq T$, because T yields two different types when evaluated twice.

The risk is that if the type system does not perfectly track dependency of types on values, or the presence of side-effects in type expressions, then you may end up with two incompatible values both claiming to be of type T.

The problem was noticed by Russo[1], in his thesis introducing first-class modules to Standard ML, where he gave a counterexample showing that the naive approach was unsound. The issue later reappeared in Scala[2].

Standard ML

```
(* Counterexample by Claudio Russo *)
module F = functor (X:sig val b:bool end)
  unpack
    if X.b then
      pack struct
        type t = int
        val x = 1
        fun y n = -n
      end
      as sig
        type t
        val x : t
        val y : t -> t
      end
    else
      pack struct
        type t = bool
        val x = true
        fun y b = if b then false else true
      end
      as sig
        type t
        val x : t
        val y : t -> t
      end
end
as sig
  type t
  val x : t
```

[1] Fig 7.12 on p. 270 of Types For Modules, Claudio Russo (1998)
[2] github.com/scala/bug/issues/2079 (2009)

```
  val y : t → t
end
module A = F (struct val b = true end)
module B = F (struct val b = false end)
val z = A.y B.x
```

Scala

```
// Counterexample by Vladimir Reshetnikov
trait A {
  type T
  var v : T
}

object B {
  def f(x : { val y : A }) { x.y.v = x.y.v }

  var a : A = _
  var b : Boolean = false
  def y : A = {
    if(b) {
      a = new A { type T = Int; var v = 1 }
      a
    } else {
      a = new A { type T = String; var v = "" }
      b = true
      a
    }
  }
}

// B.f(B) causes a ClassCastException
```

There are two standard fixes:

- **Syntactically restrict expressions appearing in types**

  The syntax of type expressions can be limited to constructions whose value cannot change. See *paths* in ML-family languages and *stable identifiers* in Scala.

  This can lead to some surprises, because valid syntax then depends on context. For instance, in OCaml, anonymous module parameters are possible in module definitions but not type expressions:

  OCaml

  ```
  module X = struct type t = int end
  module FX = F (X)                          (* ok *)
  type t1 = (F (X)).t                        (* ok *)
  module FX = F (struct type t = int end)    (* ok *)
  type t2 = (F (struct type t = int end)).t  (* error *)
  ```

(Anonymous module parameters also cause other surprises: see the avoidance problem (§20))

- **Restrict effects of expressions appearing in types**

  If the language statically tracks effects, either directly with a type-and-effect system or via encoding all effects in an `IO` monad, then it is possible to check that only pure expressions are used in types.

  There is an additional wrinkle here: some effect systems track mutation but allow nontermination to count as "pure". Depending on the exact system, allowing a possibly-nonterminating expression into the type language may be unsound because of a type-level version of the eventually, nothing (§6) problem.

20 The avoidance problem

Scoping • Subtyping

A type system which infers types can occasionally find itself having inferred a type that refers to something (a type, module, etc.) which is about to go out of scope. Referring to things which are no longer in scope is ill-formed, and doing it generally leads to unsoundness (see Scope escape (§24)).

So, the type system must approximate the desired type, using only what's still in scope and avoiding the names going out of scope, which is known as the *avoidance problem*. This is hard, and in most type systems there is no general way to do it, and some systems employ fragile heuristics.

An example of this due to Dreyer[1] can be found in OCaml, when a parameterised module is instantiated with an anonymous module:

OCaml

```
module type S = sig type t end
module F (X : S) = struct type u = X.t type v = X.t end
module AppF = F((struct type t = int end : S))
```

Here OCaml infers the following module signature for `AppF`:

OCaml

```
module AppF : sig type u type v end
```

The type `X.t` is no longer in scope as the anonymous module substituted for `X` has no name. So the signature is approximated by leaving the types abstract. However, if the definition of `F` is changed to an equivalent form:

OCaml

```
module G (X : S) = struct type u = X.t type v = u end
```

then the inferred module signature changes to the non-equivalent:

OCaml

```
module AppG : sig type u type v = u end
```

The approximation process fails to respect equivalences between module signatures, which is typical of heuristic solutions to the avoidance problem.

A well-behaved solution to the avoidance problem is to introduce existential types when necessary to give names to values that have gone out of scope. In the above example, that

[1] Fig 4.12 on p. 79 of Understanding and Evolving the ML Module System, Derek Dreyer (2005)

would lead to a signature like $\exists X : S.\ \{\texttt{type u} = X.\texttt{t};\ \texttt{type v} = X.\texttt{t}\}$. The details of when and how to introduce such existentials can be quite tricky, see Crary[2] for a recent approach.

Languages with subtyping and top/bottom types Any and Nothing can sometimes use a different solution to the avoidance problem: types that go out of scope are approximated as Any when used covariantly (as an output) and Nothing when used contravariantly (as an input). For instance, when the type A goes out of scope, a function of type $\mathsf{List}[A] \to \mathsf{List}[A]$ becomes $\mathsf{List}[\mathsf{Nothing}] \to \mathsf{List}[\mathsf{Any}]$.

Scala uses this approach. However, due to the presence of constraints in Scala types (a type $T[A]$ may be well-defined only for some A), it is not always valid to replace occurrences of A with Any or Nothing. This caused a bug in Scala[3], where invalid types were sometimes inferred:

Scala

```
// Counterexample by Guillaume Martres
class Contra[-T >: Null]

object Test {
  def foo = {
    class A
    new Contra[A]
  }
}
// The inferred type of foo is Contra[Nothing],
// but this isn't a legal type
```

[2] A Focused Solution to the Avoidance Problem, Karl Crary (2020)
[3] github.com/lampepfl/dotty/issues/6205 (2019)

21 A little knowledge...

Abstract types

The general principle broken by this counterexample is not soundness but *monotonicity*: given more knowledge about the program, the compiler should do a better job compiling it.

The concrete instance of this principle here is to do with abstract types: if the implementation of a previously-hidden abstract type is exposed, no program that previously typechecked should now fail to.

Violating this property does not cause crashes, but is confusing. Refactoring becomes tricky if loosening abstraction boundaries can cause programs to stop working.

This property does not hold in OCaml, due to an optimisation that uses a special representation for a record containing only floating-point numbers (which are usually boxed in OCaml). Since this representation is incompatible with the normal one, it is possible for a program to depend on the optimisation *not* being applied:

OCaml

```
module F : sig
 (* Deleting '= float' on the line below
    makes this program compile *)
 type t = float
end = struct
 type t = float
end

module M : sig
 type t
 type r = { foo : t }
end = struct
 type t = F.t
 type r = { foo : t }
end
```

22 Underdetermined recursion

Recursive types

Most type systems permit some flavour of recursive types, allowing the programmer to define, say, the type ListInt representing lists of integers. A ListInt consists of either the empty list, or a pair of an integer and a ListInt, giving the recursive equation:

$$\mathsf{ListInt} \cong 1 + \mathsf{Int} \times \mathsf{ListInt}$$

There are two ways to interpret the symbol $\cong$ above:

- **Isorecursive types**: there is an explicitly defined datatype/class ListInt, and we must use its constructors/methods to convert between ListInt and $1 + \mathsf{Int} \times \mathsf{ListInt}$. The symbol $\cong$ is read as "is isomorphic to".
- **Equirecursive types**: the types ListInt and $1 + \mathsf{Int} \times \mathsf{ListInt}$ are the same type, because ListInt is identified with the infinite expansion:

$$1 + \mathsf{Int} \times (1 + \mathsf{Int} \times (1 + (\mathsf{Int} \times \ldots)))$$

The symbol $\cong$ is read as "equals".

With equirecursive types, there can only be a single solution to a recursive equation. If I have another type A satisfying $A = 1 + \mathsf{Int} \times A$, then the infinite expansions of ListInt and A are equal, so $A = \mathsf{ListInt}$. With isorecursive types, on the other hand, there can be two different datatypes/classes with the same pattern of recursion, which the type system does not necessarily consider equal.

The assumption that recursive type equations have only a single solution can lead to unsoundness when combined with higher-kinded types. If it is possible to abstract over a type constructor F and two types A, B satisfying:

$$A = F[A]$$
$$B = F[B]$$

then type systems equirecursive types will assume that $A = B$, since they have the same infinite expansion $F[F[F[\ldots]]]$.

This is unsound for arbitrary type-level functions F because there can be multiple solutions for $X = F[X]$. For instance:

$$F[X] = X$$
$$A = \mathsf{Int}$$
$$B = \mathsf{String}$$

This problem has arisen in OCaml[1]:

[1] github.com/ocaml/ocaml/issues/6992 (2015)

OCaml

```
(* Counterexample by Stephen Dolan *)
type (_, _) eq = Eq : ('a, 'a) eq
let cast : type a b . (a, b) eq -> a -> b =
  fun Eq x -> x

module Fix (F : sig type 'a f end) = struct
  type 'a fix = ('a, 'a F.f) eq
  let uniq (type a) (type b)
    (Eq : a fix) (Eq : b fix) : (a, b) eq =
    Eq
end

module FixId = Fix (struct type 'a f = 'a end)
let bad : (int, string) eq = FixId.uniq Eq Eq
let _ = Printf.printf "Oh dear: %s" (cast bad 42)
```

23 Overdetermined recursion

Recursive types

While the previous example (Underdetermined recursion (§22)) was concerned with recursive type definitions that have multiple valid interpretations, here the problem is recursive type definitions that have no valid interpretations at all.

When type systems allow constraints on type parameters, type declarations must be checked to see that the constraints are valid. For instance, Scala correctly rejects the following type declaration:

Scala

```
type A >: String <: Int
// Error: lower bound String does not conform to upper bound Int
```

This purports to declare a type `A` which is both a subtype of `Int` and a supertype of `String`, but this would incorrectly imply that `String` was a subtype of `Int`.

However, a similar example is accepted[1], if the invalid declaration is split across two recursive definitions:

Scala

```
// Counterexample by Nada Amin
trait O { type A >: Any <: B; type B >: A <: Nothing }
val o = new O {}
def id(a: Any): Nothing = (a: o.B)
id("Boom")
```

The `Any <: B` check that is required by the declaration of `A` passes, because `Any <: A` (by the declaration of `A`) and `A <: B` (by the declaration of `B`). In effect, circular reasoning occurs here, where each half of the recursive definition is used to justify the other.

[1] github.com/scala/bug/issues/9715 (2016)

24 Scope escape

Scoping • Polymorphism

When both polymorphism (generic functions) and type inference are present, an issue known as *scope escape* can arise. Suppose that inside the scope of a variable `f` whose type is being inferred, we define a polymorphic function of type $\forall\alpha.\alpha \to \alpha$. While type-checking this function, we must ensure that we do not accidentally infer a type for `f` that mentions the polymorphic variable α, as it was not in scope when `f` was introduced.

Languages that mix type inference and polymorphic definitions must check that this does not occur:

Haskell

```haskell
{-# LANGUAGE RankNTypes #-}

idful :: (forall a. a -> a) -> (Int,String)
idful id = (id 5, id "hello")

g f = idful (\x -> let _ = f x in x)
-- GHC error:
--     Couldn't match expected type ‘a -> ’p0 with actual type ‘’p
--     because type variable ‘’a would escape its scope
```

OCaml

```ocaml
type idful = { id : 'a . 'a -> 'a }
let idful i = i.id 5, i.id "hello"

let g f = idful { id = fun x -> f x; x }
(* Error: This field value has type 'b -> 'b which is less general than
          'a. 'a -> 'a *)
```

Above, the `idful` function requires an argument of type $\forall\alpha.\alpha \to \alpha$, but the function it is given leaks its argument to `f`. Naively, we might infer f takes arguments of type α, but this is invalid as α is not in scope outside the argument to `idful`.

The consequences of accidental scope escape are often an internal error later in the compiler, when it tries to examine the invalid type that mentions an out-of-scope variable. If scope escape does not cause the compiler to crash, it is often a soundness issue: inferred types that escape their scope can cause, for instance, the creation of polymorphic references (§1).

Scope escape bugs have occurred at one time or another in almost every implementation of polymorphism (especially higher-rank polymorphism), including Haskell[1], OCaml[2], Rust[3] and Scala[4]:

[1] gitlab.haskell.org/ghc/ghc/-/issues/7194 (2012)
[2] github.com/ocaml/ocaml/issues/6744 (2015)
[3] github.com/rust-lang/rust/issues/58451 (2019)
[4] github.com/scala/bug/issues/7084 (2013)

Haskell

```haskell
-- Counterexample by Simon Peyton Jones
{-# LANGUAGE MultiParamTypeClasses, TypeFamilies, FlexibleContexts #-}

module Foo where

type family F a

class C b where {}

foo :: a -> F a
foo x = error "urk"

h :: (b -> ()) -> Int
h = error "urk"

f = h (\x -> let g :: C (F a) => a -> Int
                 g y = length [x, foo y]
             in ())
-- Causes an internal compiler error in Core Lint
```

OCaml

```ocaml
(* Counterexample by Leo White *)
let (n : 'b -> < m : 'a . ([< `Foo of int] as 'b) -> 'a >) =
  fun x -> object
    method m : 'x. [< `Foo of 'x] -> 'x = fun x -> assert false
  end;;
(* Type inference gave:
   Error: Values do not match:
          val n : ([< `Foo of int & 'a ] as 'b) -> < m : 'a0. 'b -> 'a0 >
       is not included in
          val n : ([< `Foo of int & 'a ] as 'b) -> < m : 'a0. 'b -> 'a0 >
   Note the & 'a in the argument type.
   That is the univar from 'a . ([< `Foo of int] as 'b) -> 'a.
   (Unknown whether this is a soundness issue) *)
```

Rust

```rust
// Counterexample by @WildCryptoFox
fn f<I>(i: I)
where
    I: IntoIterator,
    I::Item: for<'a> Into<&'a ()>,
{}

fn main() {
    // triggers internal compiler error
    f(&[f()]);
}
```

Scala

```scala
// Counterexample by Nada Amin
object Test {
  trait A { a =>
    type X
    val x: a.X
  }
  val a = new A {
    type X = Int
    val x = 1
  }
  def f(arg: A): arg.X = arg.x
  val x = f(a: A)
  // Inferred type of x references out-of-scope 'arg'
  // (Still unknown whether this is a soundness issue)
}
```

Scope escape bugs are notably absent from Idris 2[5], a self-hosting dependently-typed language whose compiler encodes scoping in the types it uses to manipulate programs, so that the compiler successfully builds only if it does not have any scope escape errors.

[5] idris2.readthedocs.io/en/latest/implementation/overview.html (2020)

25 Under false pretenses

Empty types › Equality

The empty type 0 has no values, so a variable $x : 0$ can never be supplied. So, any code within the scope of such a variable can never be executed. (For a lazy language like Haskell, read "underneath a case split on" instead of "within the scope of").

In such unreachable scopes, many type systems become a little strange. This strangeness is not by itself a problem, as it can only arise in unreachable code, but care must be taken to keep it contained.

For example, it is possible in such a scope to construct an expression of a equality type, "proving" that Int and String are equal:

OCaml

```
type empty = |
type (_,_) eq =
  Refl : ('a, 'a) eq
let f (x : empty) =
  let eq : (int, string) eq = match x with _ -> . in
  ...
```

Haskell

```
{-# LANGUAGE EmptyCase #-}
data Empty
data Eq a b where
  Refl :: Eq a a

f :: Empty -> ...
f x = ...
  where
    eq : Eq Int String
    eq = case x of {}
```

If use of `eq` introduces the equation Int = String, then you may end up with nonsensical expressions such as `20 - "hello"` becoming well-typed. This can interfere with compiler optimisations: the *constant folding* optimisation eagerly computes arithmetic operations whose arguments are constant, and may not be able to handle, say, subtraction of a string from an integer, even in dead code. Generally, hoisting optimisations (those which move an expression to an earlier position) must be treated carefully, as they risk moving a nonsensical computation from dead code to code that actually runs.

In type systems based on Martin-Löf's Intensional Type Theory, a more subtle issue can arise. ITT-based systems have a relation $A \equiv B$ called *definitional equality* (or *judgemental equality*). Definitional equality determines which expressions are considered "obviously equal" (that is, can be substituted for each other in any context with no explicit coercion required), and part of the design of an ITT-based type system is to decide which rules can be included in $\equiv$ while keeping the relation decidable.

In the presence of an empty type 0, it is tempting to make any two functions $f, g : 0 \to A$ definitionally equal, as there is only one possible function from 0 to any type. However, as noted by McBride[1], this choice breaks decidability of $\equiv$, because under the scope of a variable $x : 0$, we have:

$$\mathsf{true} \equiv (\lambda(y : 0).\mathsf{true})x \equiv (\lambda(y : 0).\mathsf{false})x \equiv \mathsf{false}$$

So, in order to decide whether $\mathsf{true} \equiv \mathsf{false}$, we would first need to decide whether the type of any variable in scope is or can be converted to 0, which is not in general decidable.

[1] Grins from my Ripley Cupboard, Conor McBride, 2009.

26 Suspicious subterms

Totality

In *total* languages, all well-typed expressions terminate and infinite loops are impossible. This property is relied upon by proof assistants like Coq and Agda: if the result of a subexpression is not needed to compute the output of a program, then the subexpression is discarded. This is an important optimisation in a proof assistant, because large parts of the program are often proofs about other parts of the program, and not directly used in computations. However, this optimisation means that any source of nontermination immediately causes a soundness issue, of the sort described in Eventually, nothing (§6).

To remain sound, these languages must check that all recursive functions eventually terminate. This is usually done by checking that when the function recurses, the argument it passes itself is a strictly smaller part of the argument it received, making infinite recursion impossible.

The tricky part of this is the *subterm check*, which determines whether one expression a strictly smaller part (a "subterm") of another. Coq, in particular, has an extensive subterm check, accepting as subterms of `u` not just direct subterms but also:

- `match ... with p1 -> c1 | p2 -> c2 | ... end`

  if each possible result `cN` is a subterm of `u`

- `fun x => c`

  if the result `c` is a subterm of `u`

This resulted in unsoundness when combined with *propositional extensionality*, an assumption widely used and available in the Coq standard library, although not made by default. Propositional extensionality states that two propositions which are equivalent are also equal: roughly, this states that the only information in a proposition carries is whether or not it is true. The details of proofs can be discarded, identifying all true propositions with each other, and likewise for false ones.

One particular consequence of this is that `False -> False` and `True` become equal, as both are true. This led to an unsoundness in Coq[1]:

Coq

```
(* Counterexample by Maxime Dénès *)
Require Import ClassicalFacts.

Section func_unit_discr.
Hypothesis Heq : (False -> False) = True.
Fixpoint contradiction (u : True) : False :=
  contradiction (
    match Heq in (_ = T) return T with
    | eq_refl => fun f:False => match f with end
    end
```

[1] sympa.inria.fr/sympa/arc/coq-club/2013-12/msg00119.html (coq-club mailing list), Maxime Dénès (2013)

```
  ).
End func_unit_discr.

Lemma foo : provable_prop_extensionality -> False.
Proof.
  intro; apply contradiction.
  apply H.
  trivial.
  trivial.
Qed.
```

The issue is that Coq's subterm check accepted the argument above as a subterm of `u : True`, which should not have any subterms. By Coq's definition of "subterm", the expression `match f with end` is a subterm of `u` because each of its zero possible results is a subterm of `u`. The fix was to restrict the subterm check in the presence of dependent matches, like the outer `match`.

Agda's subterm checker has also been confused by dependent matching on type equalities, as this counterexample shows[2]:

Agda

```
-- Andreas Abel, 2014-01-10
-- Code by Jesper Cockx and Conor McBride and folks from the Coq-club

{-# OPTIONS --without-K #-}

-- An empty type.

data Zero : Set where

-- A unit type as W-type.

mutual
  data WOne : Set where wrap : FOne -> WOne
  FOne = Zero -> WOne

-- Type equality.

data _<->_ (X : Set) : Set -> Set where
  Refl : X <-> X

-- This postulate is compatible with univalence:

postulate
  iso : WOne <-> FOne

-- But accepting that is incompatible with univalence:

noo : (X : Set) -> (WOne <-> X) -> X -> Zero
noo .WOne Refl (wrap f) = noo FOne iso f
```

[2] github.com/agda/agda/issues/1023 (2015)

```
-- Matching against Refl silently applies the conversion
-- FOne -> WOne to f.  But this conversion corresponds
-- to an application of wrap.  Thus, f, which is really
-- (wrap f), should not be considered a subterm of (wrap f)
-- by the termination checker.
-- At least, if we want to be compatible with univalence.

absurd : Zero
absurd = noo FOne iso (\ ())
```

Finally, both Coq and Agda had a related soundness issue using coinductive rather than inductive types. Coinductive types contain possibly-infinite values, and instead of the subterm check (which verifies that recursive functions do not loop infinitely when consuming a value) they use the *guardedness check* (which verifies that recursive definitions do not loop infinitely when producing a value).

As above, the guardedness checker was confused by dependent pattern matching on type equalities arising from propositional extensionality, and permitted definitions that looped unproductively[3][4]:

Coq

```
(* Counterexample by Maxime Dénès *)
CoInductive CoFalse : Prop := CF : CoFalse -> False -> CoFalse.

CoInductive Pandora : Prop := C : CoFalse -> Pandora.

Axiom prop_ext : forall P Q : Prop, (P<->Q) -> P = Q.

Lemma foo : Pandora = CoFalse.
  apply prop_ext.
  constructor.
  intro x; destruct x; assumption.
  exact C.
Qed.

CoFixpoint loop : CoFalse :=
  match foo in (_ = T) return T with eq_refl => C loop end.

Definition ff : False := match loop with CF _ t => t end.
```

Agda

```
-- Counterexample by Andreas Abel
open import Common.Coinduction
open import Common.Equality

prop = Set
```

[3] sympa.inria.fr/sympa/arc/coq-club/2014-02/msg00215.html (coq-club mailing list), Maxime Dénès (2014)
[4] sympa.inria.fr/sympa/arc/coq-club/2014-03/msg00020.html (coq-club mailing list), Andreas Abel (2014)

```
data False : prop where

data CoFalse : prop where
  CF : False → CoFalse

data Pandora : prop where
  C : ∞ CoFalse → Pandora

postulate
  ext : (CoFalse → Pandora) → (Pandora → CoFalse) → CoFalse ≡ Pandora

out : CoFalse → False
out (CF f) = f

foo : CoFalse ≡ Pandora
foo = ext λ({ (CF ()) }) λ({ (C c) → CF (out ♭( c))})

loop : CoFalse
loop rewrite foo = C ♯( loop)

false : False
false = out loop
```

27 There's only one Leibniz

Equality

Leibniz's characterisation of what it means for A to equal B is that if $P(A)$ holds for any property P, then so does $P(B)$. In other words, equality is the relation that allows substitution of B for A in any context.

There can only be one reflexive relation with this substitution property. If we have two substitutive reflexive relations $\sim$ and $\simeq$, then as soon as $A \sim B$ we can substitute the second occurrence of A for B in $A \simeq A$, so necessarily $A \simeq B$ too.

Haskell[1] had a soundness issue that arose from trying to have two distinct equality-like relations[2]. Haskell supports `newtype`, a mechanism for creating a new copy T' of a previously-defined type T, and the `GeneralizedNewtypeDeriving` mechanism allowed any typeclass that had been implemented for T to be automatically derived for T'.

`GeneralizedNewtypeDeriving` was therefore a form of substitution in an arbitrary context, which was in conflict with the equality relation in the rest of Haskell's type system, which considered T and T' distinct. In particular, type families are allowed to distinguish T and T', leading to this counterexample:

Haskell

```haskell
-- Counterexample by Stefan O'Rear
{-# OPTIONS_GHC -ftype-families #-}
{-# LANGUAGE GeneralizedNewtypeDeriving #-}
data family Z :: * -> *

newtype Moo = Moo Int

newtype instance Z Int = ZI Double
newtype instance Z Moo = ZM (Int,Int)

newtype Moo = Moo Int deriving(IsInt)
class IsInt t where
    isInt :: c Int -> c t
instance IsInt Int where isInt = id
main = case isInt (ZI 4.0) of
         ZM tu -> print tu -- segfaults
```

The fix in Haskell was to introduce *roles*[3], implicitly annotating each type and type parameter according to whether it was used in a context allowing $T = T'$ (the *representational role*, allowing efficient coercions between newtypes and their underlying types) or an a context assuming $T \neq T'$ (the *nominal role*, used in type families). For more on roles, see the GHC wiki[4].

[1] ghc.haskell.org/trac/ghc/ticket/1496 (2007)
[2] The characterisation of this issue as two conflicting equalities is due to Conor McBride
[3] Safe Zero-cost Coercions for Haskell (ICFP '14), Joachim Breitner, Richard A. Eisenberg, Simon Peyton Jones, Stephanie Weirich (2014)
[4] gitlab.haskell.org/ghc/ghc/-/wikis/roles

28 Intersecting references

Mutation · Subtyping

Intersection types are types of the form $A \wedge B$ which contain those values that are both of type A and of type B. Given such a value, it can be used as either one of the types:

$$\frac{\Gamma \vdash e : A \wedge B}{\Gamma \vdash e : A} \qquad \frac{\Gamma \vdash e : A \wedge B}{\Gamma \vdash e : B}$$

Languages differ in how intersection types may be constructed. The first style is that an expression has type $A \wedge B$ if it has both types, possibly by separate derivations:

$$\frac{\Gamma \vdash e : A \qquad \Gamma \vdash e : B}{\Gamma \vdash e : A \wedge B}$$

The second style is to use subtyping, and define $A \wedge B$ as a meet in the subtyping order, so that:

$$X \leq A \wedge B \quad \text{iff} \quad X \leq A \text{ and } X \leq B$$

In this style, an expression e can be given type $A \wedge B$ by first typechecking it with some type X which is a subtype of both, and then using the above rule and subtyping to give it type $A \wedge B$.

The crucial difference is that in the first style, the two typing derivations for $\Gamma \vdash e : A$ and $\Gamma \vdash e : B$ may be entirely different. This brings surprising complexity: for instance, in both the Coppo-Dezani[1] and Barendregt-Coppo-Dezani[2] systems (both simply typed lambda calculi with intersection types), type checking is undecidable as a term is typeable iff its execution terminates!

The counterexample here, however, is about the interaction between these types and mutability, a phenomenon pointed out by Davies and Pfenning[3]. First, a naive implementation of ML-style references is unsound:

ML

```
(* Counterexample by Rowan Davies and Frank Pfenning *)
(* Suppose we have types nat and pos,
   where nat is nonnegative integers
   and pos is its subtype of positive integers *)
let x : nat ref ∧ pos ref = ref 1 in
let () = (x := 0) in   (* typechecks: x is a nat ref *)
let z : pos = !x in    (* typechecks: x is a pos ref *)
z : pos                (* now we have the positive number zero *)
```

Despite the lack of the ∀ symbol, this is essentially the same problem as in Polymorphic references (§1): the same single reference is used at two different types. The same solutions apply, in particular, the value restriction.

[1] An extension of the basic functionality theory for the λ-calculus, M. Coppo and M. Dezani-Ciancaglini (1980)
[2] A filter lambda model and the completeness of type assignment, Barendregt, Coppo and Dezani-Ciancaglini (1983)
[3] Intersection Types and Computational Effects, Rowan Davies and Frank Pfenning (2000)

However, there is a further potential source of unsoundness. In many systems with subtyping and intersection types, it is conventional to have:

$$(A \to B) \wedge (A \to C) \leq A \to (B \wedge C)$$

That is, a value which is both a function that returns B and a function that returns C must be a function which returns values that are both B and C.

Even in the presence of the value restriction, type systems with intersection types and the above rule are unsound:

ML

```
(* Counterexample by Rowan Davies and Frank Pfenning *)
let f : (unit → nat ref) ∧ (unit → pos ref) =
  fun () → ref 1 in        (* passes the value restriction *)
let f' : unit → (nat ref ∧ pos ref) =
  f in                     (* uses the above rule *)
let x : nat ref ∧ pos ref = f ()
let () = (x := 0) in
let z : pos = !x in
z : pos                    (* same problem as before *)
```

Note that both of these counterexamples rely on the first style of intersection types above, where intersections can be introduced with two different typing derivations. They do not arise if intersections are available only through subtyping, as in the second style.

29 Polymorphic union refinement

Polymorphism • Typecase

Untagged union types are types of the form $A \vee B$ that contain values that are either of type A or of type B, with no "tag" information to denote which is which:

$$\frac{\Gamma \vdash e : A}{\Gamma \vdash e : A \vee B} \qquad \frac{\Gamma \vdash e : B}{\Gamma \vdash e : A \vee B}$$

Operationally, there is no obvious way to use an untagged union type: unlike a *disjoint union* type, which contains an additional tag bit, there is no general-purpose `match` expression that can extract either the A or the B.

For languages with runtime type introspection, some systems of *refinement types* enable consumption of union types. If $x : A \vee B$ is found via a runtime type test to not be of type A, then it may be used at type B. For instance, the following programs are valid Flow and also valid TypeScript:

TypeScript

```
function numberwang(x: number | string): string {
  if (typeof x === "number") {
    // in this block, `x: number`
    return `half of ${x * 2}`;
  } else {
    // in this block, `x: string`
    return `a string with ${x.length} characters`;
  }
}
```

TypeScript

```
class Dog {
  name: string;
  constructor(name: string) { this.name = name; }
}
class Car {
  wheels: number;
  constructor(wheels: number) { this.wheels = wheels; }
}

function description(x: Dog | Car): string {
  if (x instanceof Dog) {
    // in this block, `x: Dog`
    return `the dog ${x.name}`;
  } else {
    // in this block, `x: Car`
    return `a car with ${x.wheels} wheels`;
  }
}
```

However, runtime type tests are not always so simple. The following code also typechecks in both Flow and TypeScript, yet does not necessarily return an `Array<T>`[1]:

TypeScript

```
function orSingleton<T>(x: T | Array<T>): Array<T> {
  if (x instanceof Array) {
    return x;
  }
  return [x];
}
```

While `x instanceof Array` does imply that `x: Array<E>` for some `E`, it does not imply specifically that `x: Array<T>`, even when it is known that `x: T | Array<T>`. The problem is that `T` could itself be `Array<U>` for some type `U`, so `x: Array<U> | Array<Array<U>>` is an instance of `Array` in either case:

TypeScript

```
// Purports to be the identity function, but fails when `x` happens to
// be an array.
function id<T>(x: T): T {
  const singletonArray: T[] = orSingleton(x);
  return singletonArray[0];
}

const digits: number[] = id([3, 1, 4]);
// unsound: `digits` is actually `3`
digits.includes(4);  // TypeError: digits.includes is not a function
```

In some sense, the core issue here is the refinement itself. A runtime check that `x instanceof Array` says nothing about the element type of the array, just as a runtime check that `typeof x === "function"` says nothing about the domain or codomain. The untagged union type offers a way to exploit this flaw. The same flaw occurred in Scala[2]:

Scala

```
// Counterexample by Lionel Parreaux
def test[A]: List[Int] | A => Int =
  { case ls: List[Int] => ls.head  case _ => 0 }
test(List("oops")) // crashes
```

By contrast, Typed Racket also supports refinements of union types, but behaves correctly here:

Racket

```
#lang typed/racket
```

[1] Refinement with `instanceof` and generic unions is unsound, Flow issue #6741 (2018)
[2] github.com/lampepfl/dotty/issues/5826 (2019)

```
(: valid-refinement (All (T) (U Number (Boxof T)) (-> Number T) -> T))
(define (valid-refinement x from-number)
  (if (box? x)
      (unbox x)  ;; works: must be a `Boxof T` (good)
      (from-number x)))

(: invalid-refinement (All (T) (U T (Boxof T)) -> T))
(define (invalid-refinement x)
  (if (box? x)
      (unbox x)  ;; type error here: could be a box of something else (good!)
      x))
```

In Flow and TypeScript, this issue is resolved by using tagged (disjoint) unions instead of untagged unions:

TypeScript

```
type ElementOrArray<T> =
  {type: "ELEMENT", value: T} | {type: "ARRAY", array: T[]};

function orSingleton<T>(x: ElementOrArray<T>): T[] {
  if (x.type === "ARRAY") {
    return x.array;
  }
  return [x.value];
}

// Actually the identity function.
function id<T>(x: T): T {
  const singletonArray: T[] = orSingleton({type: "ELEMENT", value: x});
  return singletonArray[0];
}
```

Since each variant of `ElementOrArray<T>` includes a distinct `type` tag, there is no ambiguity in `orSingleton`, and the required changes to `id` fix the bug.

In JavaScript, all values are in fact tagged as one of `"string"`, `"number"`, `"object"`, or a few other options, and the `typeof` operator exposes this tag at runtime. Thus, "untagged" unions like `number | null` or `boolean | string`, where every variant has a distinct value tag, are actually tagged unions, and may still be used safely.

30 Positivity, strict and otherwise

Recursive types • Totality • Impredicativity

In a total language, type definitions that refer to themselves must be restricted:

Coq

```
-- rejected by Coq
Inductive bad := r (c : bad -> nat).
```

Agda

```
-- rejected by Agda
data Bad : Set where
  r : (Bad → ℕ) → Curry
```

Here, the type `bad` is defined recursively as consisting of a function that accepts `bad` as an input. Allowing these *negative* definitions leads to Curry's paradox (§5), and breaks totality.

The situation is more complicated if the recursive reference is underneath two function arrows:

Coq

```
-- also rejected by Coq
Inductive bad2 := r (c : (bad2 -> nat) -> nat).
```

Agda

```
-- also rejected by Agda
data Bad2 : Set where
  r : ((Bad2 → ℕ) → ℕ) → Bad2
```

This is not negative recursion: `bad2` is not defined in terms of functions that accept `bad2` values as an input, but in terms of functions that may provide `bad2` values to their argument. This is said to be positive recursion (since all recursive references to `bad2` occur to the left of an *even* number of function arrows), but not *strictly positive* (wherein all recursive references occur to the left of *zero* function arrows).

Recursive definitions which are positive yet not strictly positive can cause issues, as pointed out by Coquand and Paulin[1]. Their counterexample was translated into modern Coq by Sjöberg[2], and reproduced here:

Coq

```
(* Counterexample by Thierry Coquand and Christine Paulin
   Translated into Coq by Vilhelm Sjöberg *)
```

[1] Section 3.1 of "Inductively defined types", Thierry Coquand and Christine Paulin, 1988.
[2] Why must inductive types be strictly positive?, Vilhelm Sjöberg (2015)

```
(* Phi is a positive, but not strictly positive, operator. *)
Definition Phi (a : Type) := (a -> Prop) -> Prop.

(* If we were allowed to form the inductive type
     Inductive A: Type :=
       introA : Phi A -> A.
   then among other things, we would get the following. *)
Axiom A : Type.
Axiom introA : Phi A -> A.
Axiom matchA : A -> Phi A.
Axiom beta : forall x, matchA (introA x) = x.

(* In particular, introA is an injection. *)
Lemma introA_injective : forall p p', introA p = introA p' -> p = p'.
Proof.
  intros.
  assert (matchA (introA p) = (matchA (introA p'))) as H1 by congruence.
  now repeat rewrite beta in H1.
Qed.

(* However, ... *)

(* Proposition: For any type A, there cannot be an injection
   from Phi(A) to A. *)

(* For any type X, there is an injection from X to (X->Prop),
   which is λxλ.(y.x=y) . *)
Definition i {X:Type} : X -> (X -> Prop) :=
  fun x y => x=y.

Lemma i_injective : forall X (x x' :X), i x = i x' -> x = x'.
Proof.
  intros.
  assert (i x x = i x' x) as H1 by congruence.
  compute in H1.
  symmetry.
  rewrite <- H1.
  reflexivity.
Qed.

(* Hence, by composition, we get an injection f from A->Prop to A. *)
Definition f : (A->Prop) -> A
  := fun p => introA (i p).

Lemma f_injective : forall p p', f p = f p' -> p = p'.
Proof.
  unfold f. intros.
  apply introA_injective in H. apply i_injective in H. assumption.
Qed.

(* We are now back to the usual Cantor-Russel paradox. *)
(* We can define *)
Definition P0 : A -> Prop
```

```
  := fun x =>
       exists (P:A->Prop), f P = x /\ ~ P x.
  (* i.e., P0 x := x codes a set P such that ∉xP. *)

Definition x0 := f P0.

(* We have (P0 x0) iff ~(P0 x0) *)
Lemma bad : (P0 x0) <-> ~(P0 x0).
Proof.
split.
  * intros [P [H1 H2]] H.
    change x0 with (f P0) in H1.
    apply f_injective in H1. rewrite H1 in H2.
    auto.
  * intros.
    exists P0. auto.
Qed.

(* Hence a contradiction. *)
Theorem worse : False.
  pose bad. tauto.
Qed.
```

This counterexample uses three ingredients: non-strictly-positive definitions, impredicativity (the ability for definitions of terms in `Prop` to quantify over all of `Prop`) and a universe type (the ability to refer to `Prop` itself as a type). It appears that all three are necessary:

- The Calculus of Inductive Constructions, upon which Coq is based, is total, and has an impredicative `Prop` and a universe type for `Prop`, but requires all inductive definitions to be strictly positive.
- System F is impredicative, and can encode (or be extended with) non-strictly-positive inductive types while remaining total (see Berger et al.[3] for an example), but lacks a universe type.
- The combination of non-strictly-positive inductive types and universe types is an unusual one, but poses no theoretical problems in the absence of impredicativity. See for instance the constructions of Abel[4] or Blanqui[5].

[3] Martin Hofmann's Case for Non-Strictly Positive Data Types, Ulrich Berger, Ralph Matthes and Anton Setzer (2018)
[4] Section 7.1 of A Semantic Analysis of Structural Recursion, Andreas Abel (1999)
[5] Inductive types in the Calculus of Algebraic Constructions, Frédéric Blanqui (2006)

31 Nearly-universal quantification

Polymorphism • Subtyping

This issue is about an interaction between polymorphism and *constrained type constructors*. These are a feature of several type systems allowing the definition of a type constructor $T[-]$, where $T[A]$ is a valid type only for certain A.

This feature is different from GADTs (indexed types), which allow the definition of a type constructor $T[-]$ where $T[A]$ is always a valid type, but one which is only inhabited for certain A.

To explain this distinction, here's a GADT G and a constrained type C:

OCaml

```
type 'a g = G_int : { name : string } -> int g

type 'a c = { name: string } constraint 'a = int
```

Scala

```
sealed trait G[T]
case class G_int(i: Int) extends G[Int]

class C[A >: Int <: Int](name : String)
```

The type C$[A]$ is only legal when $A = \mathsf{Int}$, whereas G$[A]$ is always valid, but is an empty type unless $A = \mathsf{Int}$:

OCaml

```
type ok = string g

type bad = string c
(* Error: This type string should be an instance of type int *)
```

Scala

```
type Ok = G[String]

type Bad = C[String]
// Error: type arguments [A] do not conform to
// class C's type parameter bounds [A >: Int <: Int]
```

With constrained types, there are two different ways to interpret a polymorphic type such as $\forall\alpha.\mathrm{C}[\alpha] \to \mathrm{String}$:

1. $\forall\alpha.\ \ldots$ really means "for all α", so the type above is invalid since C$[\alpha]$ is not a valid type for all α.

2. $\forall\alpha.\ \ldots$ means "for all α for which the right-hand side is valid", so the type above is valid, but can only be used with $\alpha = \mathsf{Int}$.

Both OCaml and Scala choose option (1):

OCaml

```
let poly : 'a . 'a c -> string =
  fun _ -> "hello"
(* Error: The universal type variable 'a cannot be generalized:
          it is bound to int. *)
```

Scala

```
def poly[A](x: C[A]): String = "hello"
// Error: type arguments [A] do not conform to
// class C's type parameter bounds [A >: Int <: Int]
```

Option (2) requires slightly less annotation by the programmer, but it can be easy to lose track of the implicit constraints on α. Rust chooses this option, which led to a tricky bug[1].

In Rust, the type `&'a T` means a reference to type `T` which is valid for lifetime `'a`. Lifetimes are ordered by a subtyping relation `'a: 'b` (pronounced "`'a` outlives `'b`"), if the lifetime `'a` contains all of `'b`. References to references `&'a &'b T` are valid, but only if `'b: 'a` as the reference's lifetime must not outlive its contents.

There is also a longest lifetime `'static`, such that `'static: 'a` for any lifetime `'a`. The bug allows any reference to be converted to a `'static` reference, breaking soundness guarantees:

Rust

```
// Counterexample by Aaron Turon
static UNIT: &'static &'static () = &&();

fn foo<'a, 'b, T>(_: &'a &'b (), v: &'b T) -> &'a T { v }

fn bad<'c, T>(x: &'c T) -> &'static T {
    let f: fn(&'static &'c (), &'c T) -> &'static T = foo;
    f(UNIT, x)
}
```

The issue is that the type of `foo` uses `&'a &'b ()`, which is a well-formed type only if `'b: 'a`, as a reference cannot outlive its contents. This constraint `'b: 'a` is relied upon, to allow `v` to be returned.

Since Rust uses option (2) above, this constraint does not appear explicitly in the type of `foo`:

Rust

```
fn foo<'a, 'b, T> (_: &'a &'b (), v: &'b T) -> &'a T
```

[1] github.com/rust-lang/rust/issues/25860 (2015)

Contravariance allows arguments to be passed with a longer lifetime than required by the function, allowing `foo` to be used at the following type, where one argument lifetime has been extended to `'static`:

Rust

```
fn foo<'a, 'b, T> (_: &'a &'static (), v: &'b T) -> &'a T
```

However, the implicit constraint `'b: 'a` has been lost, as validity now requires only that `'static: 'a`, which is always true. This allows the type to be instantiated with `'a := 'static`, `'b := 'c`:

Rust

```
fn foo<T> (_: &'static &'static (), v: &'c T) -> &'static T
```

which is unsound.

Index and Glossary

Every entry is tagged with the type system features it involves. The features are described below, each one followed by the counterexamples in which it appears.

Polymorphism

The word “polymorphism” can refer to several different things. Here, it means “parametric polymorphism”: types like $\forall\alpha.\ \alpha \to \alpha$, allowing the same value to be used at many possible types, parameterised by a type variable. This feature is sometimes called “generics”.

Other meanings include “subtype polymorphism” (see subtyping) and “ad-hoc polymorphism” (see overloading).

Subtyping

Subtyping allows a value of a more specific type to be supplied where a value of a more general type was expected, without the two types having to be exactly equal.

Overloading

An *overloaded function* has several different versions all with the same name, where the language picks the right one to call by examining the types of its arguments at each call site.

Recursive types

Recursive types are types whose definition refers to themselves, either by using their own name during their definition, or by using explicit fixpoint operators like μ-types.

Variance

Types that take parameters (like List[A]) may have subtyping relationships that depend on the subtyping relationships of their parameters: for instance, List[A] is a subtype of List[B] only if A is a subtype of B. The manner in which the parameter's subtyping affects the whole type's subtyping is called *variance*.

Mutation

The presence of mutable values (reference cells, mutable arrays, etc.) in a language means that there are expressions which, when evaluated twice, yield different values both times (which can have consequences for the type system).

Scoping

When types are defined locally to a module, function or block, the compiler must check they do not accidentally leak out of their scope.

Typecase

Typecase refers to any runtime test that checks types. Several other names for this feature exist: `instanceof`, downcasting, matching on types.

Empty types

An empty type is a type that has no values, and can represent the return type of a function that never returns or the element type of a list that is always empty. These are not to be confused with *unit types* like C's `void` or Haskell's `()`, which are types that have a single value (and consequently carry no *information*).

Equality

Determining whether two types are equal is a surprisingly tricky business, especially in a language with advanced type system features (e.g. dependent types).

Injectivity

A parameterised type like $\mathsf{List}[A]$ is said to be *injective* if $\mathsf{List}[A] = \mathsf{List}[B]$ implies $A = B$. All, some or none of a language's parameterised types may have this property.

Totality

In a *total* language, all programs terminate, and unbounded recursion or infinite looping is impossible. Enforcing this property places a significant extra burden on the type checker.

Abstract types

An abstract type is one whose implementation is hidden: the type may in fact be implemented directly as another type, but this fact is not exposed.

A little knowledge... 62

Impredicativity

A type system is *predicative* if definitions can never be referred to, even indirectly, before they are defined. In particular, polymorphic types $\forall\alpha.\ldots$ are predicative only if α ranges over types not including the polymorphic type being defined. Predicative systems usually have restricted polymorphism (in $\forall\alpha.\ldots$, α may range only over types that do not themselves use $\forall$, or there may be a system of stratified levels of $\forall$-usage). One hallmark of impredicative systems is unrestricted $\forall$ (present in e.g. System F).

Positivity, strict and otherwise 81

Note on this edition

The text and the programs are Stephen Dolan's, taken from the sources of counterexamples.org. The arrangement is not. The web version shows alternative renderings of a counterexample one at a time, behind a row of language tabs; here they are all printed, one after another, each with its language named. The index and glossary, which stands at the front of the web version in lieu of a table of contents, has moved to the back, and a table of contents has taken its place. Sources that were links in the running text are footnotes. A few typographical slips in the prose have been corrected, and one cross reference that pointed at a page which does not exist now points at the entry it meant.

Set in Libertinus, with program text in DejaVu Sans Mono, and typeset with LuaLaTeX.